

시간 인덱스와 추세·변동성

시각을 이름표로 삼고, 잡음 속에서 신호를 꺼냅니다

`to_datetime set_index sort_index .loc
resample interpolate rolling().mean() rolling().std()`

2026년 8월 27일 (목) · 27일차 · 64차시

교안 20_01 시계열 데이터의 이해(실습 5~8) · 20_02 시간 인덱스와 집계 · 21_01 추세와 변동성

국립금오공과대학교 컴퓨터공학부 박민호

교안	다룬 범위	상태
20_01 시계열 데이터의 이해	실습 5~8 — 엔진 정렬 · 추세 방향 · 스케일 문제	완주
20_02 시간 인덱스와 집계	to_datetime · set_index · resample · 보간 · 실습 1~8	완주
21_01 추세와 변동성	이동평균 · 이동표준편차 · $\pm 2\sigma$ 밴드 · 실습 1~7	완주
21_02 변화량과 주기 분석	주기성 개념만 짧게 소개	다음 시간

이 책을 읽는 법

뱃지 · 코드블록 · 이전 회차와의 연결

1. 분류 뱃지

오늘은 **메서드**가 유난히 많습니다. 데이터프레임이나 Series에 붙어 동작하는 것들이라 앞에 점(.)이 붙습니다. 이 구분이 안 되면 오류 메시지를 읽을 수 없습니다.

뱃지	무엇	판별 신호	오늘의 예
함수	라이브러리가 제공하는 함수	앞에 모듈 이름이 옵니다	<code>pd.to_datetime()</code> <code>pd.date_range()</code>
메서드	객체에 붙어 동작하는 함수	점 뒤에 있고 괄호가 있습니다	<code>.set_index()</code> <code>.resample()</code>
속성	객체가 가진 값	점 뒤에 있고 괄호가 없습니다	<code>df.index</code> <code>s.dtype</code>
키워드	함수에 넘기는 이름 붙은 인자	등호 왼쪽에 옵니다	<code>window=</code> <code>freq=</code>
개념	문법이 아닌 사고 틀	코드로 쓸 수 없습니다	추세 · 변동성 · 노이즈 · 평활화

개념

오늘 특히 헷갈리는 것이 `.dt`와 `.rolling()`입니다. `.dt`는 **괄호가 없는 접근자**라 `.dt.year`처럼 뒤에 속성이 또 붙고, `.rolling()`은 **괄호가 있는 메서드**라 `.rolling(20).mean()`처럼 뒤에 집계 함수가 또 붙습니다. 둘 다 혼자서는 아무 일도 안 하고 뒤에 뭔가를 붙여야 결과가 나온다는 공통점이 있습니다.

2. 코드블록 읽는 법

회색 배경에 왼쪽 파란 바가 붙은 상자는 **파이썬 코드**입니다. 가로줄 아래 초록색 ▶ 줄은 **실제로 실행했을 때 나온 출력**입니다. 이 책의 모든 숫자는 실행 결과를 그대로 옮긴 것입니다.

예시

```
import pandas as pd
```

```
df = pd.read_csv("Data/20_engine01_timestamp_sample.csv")
print(df.shape)
print(df["timestamp"].dtype)
```

```
▶ (182, 6)
▶ object
```

주의

오늘 코드에는 **한글 변수 이름**이 자주 나옵니다. `daily_7`, `전반_5`처럼 실습 노트북에 쓰인 이름을 그대로 옮겼기 때문입니다. 파이썬은 한글 변수명을 허용하지만, **실무 코드에서는 영문을 쓰는 편이 안전합니다**. 인코딩 문제나 협업 시 혼란이 생길 수 있어서입니다.

3. 오늘 쓰는 데이터 두 가지

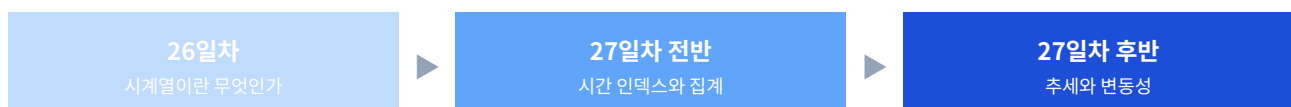
파일	크기	시간 축	쓰는 곳
21_cmapss_fd001_sample.csv	1031행 10열	cycle (작동 회차)	STEP 1 · STEP 6~7
20_engine01_timestamp_sample.csv	182행 6열	timestamp (실제 시각)	STEP 2~5

개념

오늘 수업은 **cycle → timestamp → cycle**이라는 왕복 구조입니다. 어제 배운 cycle 데이터로 추세를 마무리하고(STEP 1), timestamp 데이터로 넘어가 **시각을 이름표로 쓰는 법**을 배운 뒤(STEP 2~5), 다시 cycle로 돌아와 **이동평균과 이동표준편차**를 다룹니다(STEP 6~7). 두 시간 축을 오가며 **어떤 시계열에도 알맞은 도구를 고르는 힘**을 기르는 구성입니다.

4. 어제와 오늘의 연결

어제는 시계열이 **무엇인지**를 배웠습니다. 오늘은 그것을 **어떻게 다루는지**를 배웁니다. 개념에서 도구로 넘어가는 날입니다.



어제 배운 것	오늘 그것으로 하는 일
시계열은 순서가 정보다	<code>sort_index()</code> · <code>sort_values()</code> 로 순서를 보장합니다
timestamp와 cycle 두 가지 시간 축	<code>to_datetime()</code> 으로 cycle 데이터를 timestamp로 바꿔 씁니다
추세 · 변동성 · 이상 징후 세 신호	<code>rolling().mean()</code> 과 <code>rolling().std()</code> 로 앞의 두 개를 직접 잡니다
C-MAPSS 엔진 5대, 센서 21개	학습용으로 센서 5개만 남긴 <code>21_cmapss</code> 샘플을 씁니다

연결

어제 STEP 8에서 "추세가 가장 먼저 잡히는 약한 신호, 변동성이 중간, 이상치가 가장 강한 신호"라고 배웠습니다. 오늘 STEP 6에서 추세를 재는 도구를, STEP 7에서 변동성을 재는 도구를 배웁니다. 어제의 개념 지도가 오늘 그대로 실행됩니다.

CONTENTS

목차

오늘 배우는 것을 한눈에

장	제목	무엇을 배우나
CH 0	오늘의 지도	왜 시각을 이름표로 만드는가 · 준비 3단계
STEP 1	엔진 하나의 추세 읽기	정렬 · 첫글 비교 · 스케일 문제 · 20_01 실습 5~8
STEP 2	문자열 날짜를 시간으로	to_datetime · dtype · .dt 조각 추출 · 실습 1·2
STEP 3	시간 인덱스 만들기	set_index · sort_index · 부분 문자열 인덱싱 · 실습 3·4
STEP 4	리샘플링으로 묶기	resample · 빈도 문자열 · 집계 함수 · asfreq · 실습 5·6
STEP 5	추세 확인과 업샘플링	일별 평균 · 결측 발생 · interpolate · 실습 7·8
STEP 6	노이즈와 이동평균	rolling().mean() · 윈도우 크기 · 실습 1~4
STEP 7	변동성 분석	rolling().std() · $\pm 2\sigma$ 밴드 · 점증과 급증 · 실습 5~7
부록 A	치트시트	오늘 나온 모든 도구를 한 표에
부록 B	오류·함정 사전	자주 걸리는 함정 · 결론이 틀리는 경우
부록 C	실습 하이라이트와 다음 시간	핵심 결론 정리 · 21_02 예고

개념

오늘의 한 문장. **"평균은 어디로 가는지, 표준편차는 얼마나 불안한지를 말한다."** 두 신호는 별개입니다. 오늘 데이터에서 다섯 센서 모두 추세는 뚜렷한데($|r|$ 0.837 이상), 변동성이 커지는 것은 그중 둘뿐입니다. **추세만 보면 다 같아 보이지만 변동성을 보면 위험한 센서가 갑니다.**

오늘의 지도

시각을 이름표로 만드는 이유

0-1. 문자열 날짜의 문제

CSV에서 막 읽은 날짜는 **눈에는 날짜로 보여도 컴퓨터에게는 그냥 글자**입니다. 그 상태에서는 시간으로 할 수 있는 일이 거의 없습니다.

하고 싶은 일		
인식	글자 덩어리 (object)	시간 값 (datetime64)
3일 뒤 날짜 계산	불가능	가능
특정 기간만 자르기	불가능	가능
시간 순 정렬	어긋남 — 03-10이 03-10보다 뒤	정확
시·일·월 단위 집계	불가능	가능

비유

엑셀에서 날짜가 글자로 인식돼 정렬이 이상해지는 경험과 **똑같은 문제**입니다. 화면에 날짜처럼 보인다고 컴퓨터도 날짜로 아는 것이 아닙니다. **읽어 들인 직후에 반드시 자료형부터 확인**해야 합니다.

0-2. 시간 분석 준비 3단계



세 줄이면 시계열 준비가 끝납니다

```
import pandas as pd

df = pd.read_csv("Data/20_engine01_timestamp_sample.csv")

# ① 글자를 시간으로
df["timestamp"] = pd.to_datetime(df["timestamp"])

# ② 시간 열을 인덱스로 (결과를 반드시 다시 담기)
df = df.set_index("timestamp")

# ③ 시각 오름차순 정렬
df = df.sort_index()

print(type(df.index).__name__)
▶ DatetimeIndex
```

개념

이 세 단계는 한 번만 해 두면 이후 분석 내내 효과가 유지됩니다. 그리고 순서가 정해져 있습니다. 변환하지 않고 set_index부터 하면 인덱스가 문자열인 채로 올라가 시간 기능이 작동하지 않습니다. 정렬하지 않으면 시간 자르기에서 빈 결과나 오류가 납니다. **변환 → 인덱스 → 정렬**을 한 세트로 외우십시오.

0-3. 시간 인덱스가 열어 주는 세 가지

가능해지는 일	문법	예
① 시각으로 행 선택	<code>.loc[시각]</code>	<code>df.loc["2024-03-01 09:00:00"]</code>
② 기간 자르기	<code>.loc[시작:끝]</code>	<code>df.loc["2024-03-05":"2024-03-07"]</code>
③ 시간 단위 집계	<code>.resample()</code>	<code>df["temp_out"].resample("1D").mean()</code>

비유

인덱스는 각 행에 붙은 이름표입니다. CSV를 막 읽으면 0·1·2 같은 순번이 붙는데, 이 이름표를 실제 시각으로 바꾸는 것이 DatetimeIndex입니다. 도서관 책의 청구기호처럼, **시각만 알면 그 행을 바로 찾아낼 수 있게** 됩니다. 책의 목차가 3장이 몇 쪽인지 알려주듯 시간 인덱스는 시각으로 된 목차입니다.

0-4. cycle 데이터를 timestamp로 보는 이유

C-MAPSS는 날짜가 없는 cycle 데이터인데, 오늘은 날짜가 붙은 샘플로 연습합니다. 이유가 있습니다.

이유	설명
① 현장 데이터의 대다수	공장·발전소·건물 에너지 등 실제 설비 데이터 상당수가 timestamp 기반입니다
② cycle로는 연습 불가	날짜가 없으면 시·일·월 단위 집계를 아예 시도할 수 없습니다
③ 약속으로 변환 가능	1 cycle = 1시간 으로 약속하면 137번째 작동이 "작동 후 137시간째"가 됩니다

주의

오늘 쓰는 날짜는 **진짜 측정 시각이 아니라 연습용 가상 시각**입니다. 강사님이 1 cycle = 1시간으로 약속해 만든 것입니다. 다만 **작동 간격이 일정한 설비라면 실제로도 이 변환이 가능합니다**. 중요한 것은 `to_datetime` → `set_index` → `resample`이라는 흐름 자체는 **진짜 timestamp 데이터를 만나도 똑같다**는 점입니다.

0-5. 오늘 쓰는 두 데이터 살펴보기

1번 엔진을 시각 기준으로 바꾼 샘플

```
# ① timestamp 샘플 — STEP 2~5에서 사용
df = pd.read_csv("Data/20_engine01_timestamp_sample.csv")
print(df.shape)
print(list(df.columns))
print(df.isna().sum().to_dict())
```

```
▶ (182, 6)
▶ ['timestamp', 'temp_out', 'temp_core', 'pressure', 'vibration', 'flow']
▶ {'timestamp': 0, 'temp_out': 0, 'temp_core': 0, 'pressure': 2, 'vibration': 3, 'flow': 0}
```

센서 21개 중 5개만 남긴 학습용 샘플

```
# ② cycle 샘플 — STEP 1 · 6 · 7에서 사용
d = pd.read_csv("Data/21_cmapss_fd001_sample.csv")
print(d.shape)
print(list(d.columns))
print(d["unit_nr"].value_counts().sort_index().to_dict())
print("결측:", d.isna().sum().sum())
```

```
▶ (1031, 10)
▶ ['unit_nr', 'time_cycles', 'setting_1', 'setting_2', 'setting_3', 's_2', 's_3', 's_4', 's_7', 's_11']
▶ {1: 192, 2: 287, 3: 179, 4: 218, 5: 155}
▶ 결측: 0
```


비교 항목		
크기	182행 6열	1031행 10열
시간 축	timestamp	time_cycles
대상	1번 엔진 하나	엔진 5대
기간	2024-03-01 00시 ~ 03-08 23시	1 ~ 287 회차
결측	pressure 2 · vibration 3	없음
센서	5개 (온도2·압력·진동·유량)	5개 (s_2 s_3 s_4 s_7 s_11)

설비 적용

어제 쓴 20_cmapss 샘플은 엔진 5대가 **192 · 287 · 179 · 189 · 269**회차였는데, 오늘 21_cmapss 샘플은 **192 · 287 · 179 · 218 · 155**입니다. 4번과 5번 엔진이 다릅니다. 같은 이름처럼 보여도 다른 샘플이므로, 어제 문서의 숫자와 오늘 문서의 숫자를 섞어 쓰면 안 됩니다. 파일 이름 앞의 번호를 항상 확인하십시오.

STEP 1

엔진 하나의 추세 읽기

정렬 · 첫끝 비교 · 스케일 문제

어제 실습 4까지 하고 멈춘 지점부터 이어갑니다. 데이터가 어떻게 생겼는지는 확인했으니, 이제 **한 엔진을 골라 시간 순으로 늘어놓고 변화를 읽습니다.**

1-1. 보기 전에 정렬한다

메서드 `.sort_values()` 특정 열 기준으로 행 순서 맞추기

정의 시계열은 **순서가 생명**이라 그리기 전에 반드시 시간 축 기준으로 정렬합니다.

형태 `df[df["unit_nr"] == 2].sort_values("time_cycles")`

```
df_5 = pd.read_csv("Data/20_cmapss_fd001_sample.csv")

# 2번 엔진만 골라 작동 회차 순으로 정렬
engine_5 = df_5[df_5["unit_nr"] == 2].sort_values("time_cycles")

print(engine_5[["time_cycles", "s_11", "s_7"]].head(1))
print(engine_5[["time_cycles", "s_11", "s_7"]].tail(1))
```

	time_cycles	s_11	s_7
▶ 192	1	47.30	553.84
▶			
▶	time_cycles	s_11	s_7
▶ 478	287	48.18	546.17

왜 쓰나 엔진을 안 고르면 1번 엔진의 마지막 회차 다음에 2번 엔진의 첫 회차가 붙습니다. 그래프로 그리면 값이 뚝 떨어졌다가 다시 오르는 톱니 모양이 나와 흐름이 완전히 깨집니다.

응용 `.head(1)`이 붙은 행 번호가 **192**인 것에 주목하십시오. 원본 인덱스가 그대로 따라온 것입니다. 걸러 낸 뒤 인덱스를 새로 매기고 싶으면 `reset_index`를 씁니다.

```
# 원본 인덱스가 그대로 따라옴
print(engine_5.index[:3].tolist())

# 0부터 다시 매기고 싶을 때
e = engine_5.reset_index(drop=True)
print(e.index[:3].tolist())
```

▶ [192, 193, 194]
▶ [0, 1, 2]

개념

drop=True를 빼면 기존 인덱스가 index라는 새 열로 남습니다. 대부분의 경우 필요 없으므로 습관적으로 붙이는 편이 좋습니다. 다만 원래 위치를 추적해야 한다면 일부러 남기기도 합니다.

주의

"대상을 고르고 시간 순으로 정렬"은 거의 항상 붙어 다니는 두 단계입니다. 필터 다음에 정렬을 쓰는 것을 하나의 관용구로 외워 두십시오.

1-2. 실습 5 — 첫값과 끝값으로 방향 판정

Practice/20_01_example.ipynb — 실습 5

```
# iloc[0]은 첫 행, iloc[-1]은 마지막 행
for sensor in ["s_11", "s_7", "s_12", "s_20"]:
    start = engine_5[sensor].iloc[0]
    end = engine_5[sensor].iloc[-1]
    방향 = "상승" if end > start else "하강"
    print(f"{sensor:5} {start:>8.2f} -> {end:>8.2f}"
          f" ({방향}, 변화량 {end - start:+.2f})")
```

```
▶ s_11      47.30 -> 48.18 (상승, 변화량 +0.88)
▶ s_7       553.84 -> 546.17 (하강, 변화량 -7.67)
▶ s_12      520.52 -> 513.55 (하강, 변화량 -6.97)
▶ s_20       38.94 -> 37.94 (하강, 변화량 -1.00)
```

속성 .iloc[] 위치 번호로 행 꺼내기

정의 i는 integer, 즉 정수 위치입니다. 인덱스 라벨이 아니라 몇 번째인지로 접근합니다.

형태 series.iloc[0] · series.iloc[-1]

```
# .iloc은 위치, .loc은 라벨
print(engine_5["s_11"].iloc[0])    # 첫 번째 값
print(engine_5["s_11"].iloc[-1])   # 마지막 값

# .loc은 인덱스 라벨을 씁니다
print(engine_5["s_11"].loc[192])   # 라벨이 192인 행

▶ 47.3
▶ 48.18
▶ 47.3
```

왜 쓰나 걸러 낸 데이터프레임은 인덱스가 0부터 시작하지 않습니다. 위 예에서 첫 행의 라벨이 192이므로 `.loc[0]`은 오류가 납니다. "몇 번째"를 원하면 `.iloc`입니다.

응용 -1은 마지막, -2는 뒤에서 두 번째입니다. 파이썬 리스트와 같은 규칙이라 시계열의 끝을 볼 때 편리합니다.

```
# 마지막 5개 회차의 평균 — 최근 상태 요약
print(round(engine_5["s_11"].iloc[-5:].mean(), 3))

# 첫 5개와 비교
print(round(engine_5["s_11"].iloc[:5].mean(), 3))
```

```
▶ 48.152
▶ 47.3
```

개념

`.iloc[-5:]`처럼 **슬라이싱도 됩니다**. 첫 5개 평균 47.3과 마지막 5개 평균 48.152를 비교하면 첫값·끝값 하나씩만 보는 것보다 **훨씬 안정적인 판정**이 됩니다. 값 하나는 우연히 튼 값일 수 있기 때문입니다.

자주 하는 실수

`.loc`과 `.iloc`을 헷갈리면 `KeyError`가 나거나 **엉뚱한 행이 조용히 선택됩니다**. 필터링 직후에는 인덱스가 원본을 유지하므로 특히 주의해야 합니다.

1-3. 두 점만 보면 위험합니다

21일치의 `corr()`이 추세 판정 도구가 됩니다

```
# 첫값·끝값만 보면 우연일 수 있으니 회차와의 상관계수로 확인
for sensor in ["s_11", "s_7", "s_12", "s_20", "s_14"]:
    r = engine_5["time_cycles"].corr(engine_5[sensor])
    print(f"{sensor:5} r = {r:+.3f}")
```

```
▶ s_11  r = +0.911
▶ s_7    r = -0.966
▶ s_12  r = -0.968
▶ s_20  r = -0.951
▶ s_14  r = +0.298
```

센서	첫→끝	상관계수	판정
s_11	47.30 → 48.18	+0.911	뚜렷한 상승 추세
s_7	553.84 → 546.17	-0.966	뚜렷한 하강 추세

센서	첫→끝	상관계수	판정
s_12	520.52 → 513.55	-0.968	뚜렷한 하강 추세
s_20	38.94 → 37.94	-0.951	뚜렷한 하강 추세
s_14	—	+0.298	추세라 하기 어려움

설비 적용

time_cycles와의 상관계수 부호가 곧 추세 방향입니다. 양수면 회차가 늘수록 값이 커지는 것이고, 음수면 작아지는 것입니다. 그리고 절댓값이 클수록 그 추세가 직선에 가깝다는 뜻입니다.

s_14만 +0.298로 뚝 떨어집니다. 첫값과 끝값만 보면 방향이 있어 보여도, 전체 흐름은 방향이 뚜렷하지 않다는 뜻입니다. 두 점만 보고 판정하면 놓칠 뻔한 사실입니다.

1-4. 실습 7 — 스케일이 다르면 비교가 안 됩니다

Practice/20_01_example.ipynb — 실습 7

```
# 크기가 크게 다른 두 센서를 일부러 겹쳐 그려 보기
plt.figure(figsize=(10, 4))
plt.plot(engine_5["time_cycles"], engine_5["s_14"], label="s_14")
plt.plot(engine_5["time_cycles"], engine_5["s_20"], label="s_20")
plt.legend()
plt.show()

print(engine_5[["s_2", "s_7", "s_14", "s_20"]].describe().round(2).T)
```

	count	mean	std	min	max
s_2	287.0	641.75	0.49	640.46	642.87
s_7	287.0	549.99	2.39	545.22	554.68
s_14	287.0	8138.26	11.93	8099.12	8176.36
s_20	287.0	38.45	0.27	37.88	39.06

자주 하는 실수

s_14는 약 8138, s_20은 약 38입니다. 자릿수가 200배 넘게 차이 납니다. 한 축에 그리면 s_20은 바닥에 완전히 납작하게 눌러 변화가 전혀 보이지 않습니다.

그런데 s_14의 범위 폭은 77.24, s_20은 1.18입니다. 절대 폭은 s_14가 65배 크지만, 평균 대비로 보면 s_14가 0.95%, s_20이 3.07%로 오히려 s_20이 더 많이 변합니다. 그림만 보고 "s_20은 변화가 없다"고 판단하면 완전히 틀린 결론입니다.

해법	방법	언제
① 따로 그리기	subplots로 칸을 나눠 각각	가장 안전 — 오늘 실습 방식
② 정규화	$(x - \min) / (\max - \min)$ 으로 0~1 변환	모양만 비교하고 싶을 때
③ 이중 축	ax.twinx()	두 개까지만 — 셋 이상은 혼란

② **정규화**는 STEP 6의 실습 4에서 실제로 사용합니다. 값의 크기를 버리고 **모양만 남기는** 방법이라, 방향 비교에는 좋지만 **크기 비교에는 쓸 수 없다**는 점을 기억하십시오.

1-5. 실습 8 — 구조 미니 리포트

Practice/20_01_example.ipynb — 실습 8

```
print("=" * 50)
print("C-MAPSS FD001 샘플 데이터 구조 리포트")
print("=" * 50)
print(f"전체 크기 : {df_5.shape[0]}행 {df_5.shape[1]}열")
print(f"엔진 대수 : {df_5['unit_nr'].nunique()}대")
print(f"전체 결측 : {df_5.isna().sum().sum()}개")

std_8 = df_5.std(numeric_only=True).round(4)
상수_8 = std_8[std_8 == 0].index.tolist()
print(f"상수 컬럼 : {len(상수_8)}개")

print("-" * 50)
print(f"대상 엔진 : 2번")
print(f"작동 회차 : {len(engine_5)}회"
      f" ({engine_5['time_cycles'].min()} ~ {engine_5['time_cycles'].max()}")
```

```
▶ =====
▶ C-MAPSS FD001 샘플 데이터 구조 리포트
▶ =====
▶ 전체 크기 : 1116행 26열
▶ 엔진 대수 : 5대
▶ 전체 결측 : 0개
▶ 상수 컬럼 : 8개
▶ -----
▶ 대상 엔진 : 2번
▶ 작동 회차 : 287회 (1 ~ 287)
```

개념

분석을 시작하기 전에 이런 요약의 한 번 짚어 두는 것이 좋은 습관입니다. 나중에 결과가 이상할 때 "내가 무슨 데이터를 봤더라"를 되짚을 수 있고, 리포트에 그대로 붙일 수도 있습니다. "=" * 50은 등호를 50번 반복한 문자열을 만드는 것으로, 구분선을 그리는 관용구입니다.

1-6. 엔진 다섯 대를 나란히 놓고 보기

한 엔진만 보면 알 수 없는 것이 있습니다. 다섯 대를 나란히 놓으면 예지보전의 핵심 아이디어가 보입니다.

21_cmapss 샘플 — STEP 6에서 쓸 데이터로 미리 확인

```
# 엔진마다 첫값·끝값·상관계수를 한 번에
for u in sorted(df_5["unit_nr"].unique()):
    g = df_5[df_5["unit_nr"] == u].sort_values("time_cycles")
    첫, 끝 = g["s_4"].iloc[0], g["s_4"].iloc[-1]
    r = g["time_cycles"].corr(g["s_4"])
    print(f" {u}번 {len(g):3}회차 | s_4 {첫:.2f} -> {끝:.2f}"
          f" ({끝-첫:+.2f}) r={r:+.3f}")
```

- ▶ 1번 192회차 | s_4 1399.30 -> 1416.22 (+16.92) r=+0.989
- ▶ 2번 287회차 | s_4 1399.71 -> 1416.77 (+17.06) r=+0.989
- ▶ 3번 179회차 | s_4 1399.79 -> 1419.03 (+19.24) r=+0.988
- ▶ 4번 218회차 | s_4 1399.73 -> 1418.94 (+19.21) r=+0.988
- ▶ 5번 155회차 | s_4 1399.11 -> 1416.50 (+17.39) r=+0.987

설비 적용

다섯 대 모두 s_4가 1399 근처에서 시작해 1416~1419에서 끝납니다. 시작값도 끝값도 거의 같습니다. 그런데 거기까지 걸린 회차는 155에서 287까지 1.85배 차이가 납니다.

이것이 C-MAPSS 데이터의 설계 의도입니다. "어느 값에 도달하면 고장난다"는 공통이지만, 도달하는 속도는 개체마다 다릅니다. 그래서 예지보전은 "이 모델은 200회에 고장난다"가 아니라 "이 엔진은 지금 어디쯤 와 있는가"를 봐야 합니다.

그리고 상관계수가 다섯 대 모두 0.987 이상입니다. 개체차가 커도 열화 패턴 자체는 일관된다는 뜻이고, 이것이 예측 모델을 만들 수 있는 근거가 됩니다.

STEP 2

문자열 날짜를 시간으로

to_datetime · dtype · .dt

여기서 데이터가 바뀝니다. cycle 대신 **실제 날짜와 시각이 붙은 데이터**를 다룹니다. 첫 관문은 **글자를 시간으로 바꾸는 일**입니다.

2-1. 실습 1 — 변환 전후 자료형 확인

함수 `pd.to_datetime()` 날짜처럼 생긴 글자를 진짜 시간 값으로

정의 열을 통째로 받아 **datetime64 자료형**으로 바꿉니다. 이 변환을 거쳐야 뒤의 모든 시간 기능이 작동합니다.

형태 `df["timestamp"] = pd.to_datetime(df["timestamp"])`

```
df_1 = pd.read_csv("Data/20_engine01_timestamp_sample.csv")
print("shape:", df_1.shape)

print("변환 전 타입:", df_1["timestamp"].dtype)
print("첫 값:", df_1["timestamp"].iloc[0])

# 글자를 시간 값으로
df_1["timestamp"] = pd.to_datetime(df_1["timestamp"])

print("변환 후 타입:", df_1["timestamp"].dtype)

# 시간끼리 뺄셈이 되는지 확인
print("간격:", df_1["timestamp"].iloc[1] - df_1["timestamp"].iloc[0])
```

- ▶ shape: (182, 6)
- ▶ 변환 전 타입: object
- ▶ 첫 값: 2024-03-01 00:00:00
- ▶ 변환 후 타입: datetime64[ns]
- ▶ 간격: 0 days 01:00:00

왜 쓰나 object는 글자, datetime64는 시간입니다. **변환이 잘 됐는지는 오직 dtype으로 확인**합니다. 화면에 찍히는 모양은 변환 전후가 거의 같아 눈으로는 구별할 수 없습니다.

응용 변환이 성공했다는 가장 확실한 증거는 **시간끼리 뺄셈이 되는 것**입니다. 결과가 Timedelta 자료형으로 나옵니다.


```
# Timedelta에서 숫자를 꺼내기
gap = df_1["timestamp"].iloc[1] - df_1["timestamp"].iloc[0]
print(type(gap).__name__)
print(gap.total_seconds(), "초")

# 전체 기간
기간 = df_1["timestamp"].iloc[-1] - df_1["timestamp"].iloc[0]
print("전체 기간:", 기간)
```

```
▶ Timedelta
▶ 3600.0 초
▶ 전체 기간: 7 days 23:00:00
```

개념

datetime64[ns]의 ns는 **나노초 단위로 저장**한다는 뜻입니다. 그래서 초·분·시 어느 단위든 정확하게 계산됩니다. 다만 나노초 정밀도 때문에 표현 가능한 범위가 **1677년~2262년**으로 제한되는데, 설비 데이터에서는 문제가 되지 않습니다.

주의

read_csv에서 parse_dates=["timestamp"]를 주면 **읽으면서 바로 변환**할 수 있습니다. 다만 어느 열이 날짜인지 미리 알아야 하므로, 처음 보는 파일은 일단 읽고 dtype을 확인한 뒤 변환하는 편이 안전합니다.

2-2. 형식이 특이할 때

상황	문제	해법
모호한 형식	03-04-2024 — 3월 4일인가 4월 3일인가	format="%d-%m-%Y"
변환 불가능 값	빈 값이나 엉뚱한 글자가 섞임	errors="coerce"
여러 형식 혼재	파일마다 표기가 다름	format="mixed"

대부분은 옵션 없이도 알아서 됩니다

```
# 형식 기호 — 연도는 대문자 Y, 월 소문자 m, 일 소문자 d, 시 대문자 H
pd.to_datetime("2024-03-01", format="%Y-%m-%d")
pd.to_datetime("03-01-2024", format="%m-%d-%Y")
```

변환 실패를 NaT(Not a Time)로 처리

```
s = pd.Series(["2024-03-01", "없음", "2024-03-03"])
print(pd.to_datetime(s, errors="coerce"))
```

```
▶ 0    2024-03-01
▶ 1           NaT
▶ 2    2024-03-03
▶ dtype: datetime64[ns]
```

주의

NaT는 시간 버전의 NaN입니다. 22일차에 배운 `isna()`로 똑같이 셀 수 있습니다. `errors="coerce"`를 쓰면 오류로 멈추지 않고 조용히 NaT가 되므로, 변환 직후 반드시 `isna().sum()`으로 몇 개가 실패했는지 확인해야 합니다. 확인 없이 넘어가면 데이터 일부가 사라진 줄도 모르게 됩니다.

2-3. 실습 2 — .dt로 시간 조각 꺼내기

속성 `.dt` 변환된 시간 열에서 조각 꺼내는 접근자

정의 `dt`는 `datetime`의 줄임말입니다. 괄호가 없는 접근자이고, 뒤에 `.year` · `.month` 같은 속성을 또 붙여 씁니다.

형태 `df["timestamp"].dt.year` · `.dt.month` · `.dt.hour`

```
df_2 = pd.read_csv("Data/20_engine01_timestamp_sample.csv")
df_2["timestamp"] = pd.to_datetime(df_2["timestamp"])
```

```
print("연:", df_2["timestamp"].dt.year.iloc[0])
print("월:", df_2["timestamp"].dt.month.iloc[0])
print("일:", df_2["timestamp"].dt.day.iloc[0])
print("시:", df_2["timestamp"].dt.hour.iloc[0])
```

시 조각을 새 열로 추가

```
df_2["hour"] = df_2["timestamp"].dt.hour
print("hour 앞 세 값:", df_2["hour"].head(3).tolist())
```

```
▶ 연: 2024
▶ 월: 3
▶ 일: 1
▶ 시: 0
▶ hour 앞 세 값: [0, 1, 2]
```

왜 쓰나 시간 조각을 **파생 열**로 만들면 21일차에 배운 **groupby**를 그대로 쓸 수 있습니다. 시간대별·요일별 운영 패턴 분석이 여기서 열립니다.

응용 조각을 꺼냈으면 바로 그룹 집계로 이어집니다. **설비 작동 패턴이 시간대에 따라 다른 경우가 많기** 때문입니다.

```
# 시간대별 평균 — 파생 열 + groupby
print(df_2.groupby("hour")["temp_out"].mean().round(2).head(6))
```

```
▶ hour
▶ 0    641.67
▶ 1    641.59
▶ 2    641.70
▶ 3    641.85
▶ 4    641.82
▶ 5    641.80
▶ Name: temp_out, dtype: float64
```

개념

.dt는 **datetime 자료형에만 붙습니다**. 변환하지 않은 문자열 열에 **.dt**를 쓰면 **AttributeError: Can only use .dt accessor with datetimelike values**가 납니다. 이 오류 메시지가 뜨면 **변환을 빠뜨린** 것입니다.

주의

자주 쓰는 조각은 `.dt.year · .dt.month · .dt.day · .dt.hour · .dt.minute`, 그리고 **요일인** `.dt.dayofweek`입니다. **요일은 0이 월요일, 6이 일요일입니다. 헛갈리면 `.dt.day_name()`으로 이름을 직접 확인할 수 있습니다.**

설비 적용

시간대별 평균이 641.59~641.85 사이로 거의 평평합니다. **하루 중 특정 시간대에 온도가 튀는 패턴은 없다는** 뜻입니다. 만약 새벽 시간대만 값이 낮았다면 외기 온도의 영향을, 특정 교대 시간에만 튀었다면 운전 방식의 차이를 의심해야 합니다. **평평한 것도 결론입니다.**

2-4. 요일 조각으로 한 번 더

시간대에서 아무것도 안 나왔다면 **다른 조각으로 바꿔** 봅니다. 요일은 교대 근무나 주말 운전 패턴을 잡아내는 데 유용합니다.

요일별로 묶어 보기

```
# dayofweek는 0이 월요일, 6이 일요일
df_2["dow"] = df_2["timestamp"].dt.dayofweek
df_2["dname"] = df_2["timestamp"].dt.day_name()
```

```
print(df_2.groupby(["dow", "dname"])["temp_out"]
      .agg(["count", "mean"]).round(2))
```

		count	mean
▶ dow	dname		
▶ 0	Monday	22	641.68
▶ 1	Tuesday	24	641.84
▶ 2	Wednesday	22	642.10
▶ 3	Thursday	24	642.20
▶ 4	Friday	46	641.74
▶ 5	Saturday	20	641.34
▶ 6	Sunday	24	641.56

자주 하는 실수

금요일만 46건입니다. 다른 요일의 두 배입니다. 데이터 기간이 3월 1일부터 8일까지 8일간이라 **금요일이 두 번 (3/1과 3/8)** 들어갔기 때문입니다.

이걸 모르고 "금요일에 측정이 많다"고 해석하면 완전히 틀린 결론입니다. **count**를 항상 함께 보십시오. **그룹별 개수가 고르지 않으면 평균 비교의 신뢰도도 달라집니다.**

그리고 요일별 평균은 641.34~642.20으로, 이것도 **요일 효과가 아니라 8일간의 상승 추세**가 요일에 묻어난 것뿐입니다. 수·목이 높은 이유는 그날이 중간이라서가 아니라 **날짜가 뒤쪽**이라서입니다.

자주 쓰는 `.dt` 조각을 정리해 둡니다. **모두 괄호 없는 속성**이고, `day_name()`만 예외로 메서드입니다.

조각	꺼내는 것	값의 범위	쓰는 곳
<code>.dt.year</code>	연도	2024	연도별 비교
<code>.dt.month</code>	월	1 ~ 12	계절성 분석
<code>.dt.day</code>	일	1 ~ 31	일자 기준 분석
<code>.dt.hour</code>	시	0 ~ 23	시간대별 운전 패턴
<code>.dt.minute</code>	분	0 ~ 59	분 단위 로그
<code>.dt.dayofweek</code>	요일 번호	0(월) ~ 6(일)	교대·주말 패턴
<code>.dt.day_name()</code>	요일 이름	Monday ~ Sunday	사람이 읽을 표
<code>.dt.date</code>	날짜만	2024-03-01	일별 그룹핑

주의

`.dt.dayofweek`는 **0이 월요일**입니다. 파이썬 표준 라이브러리와 같은 규칙이지만, 엑셀이나 다른 언어에서는 일요일이 0인 경우도 있어 **헷갈리기 쉽습니다**. 확신이 안 서면 `.dt.day_name()`으로 이름을 함께 찍어 확인하십시오.

STEP 3

시간 인덱스 만들기

set_index · sort_index · 부분 문자열 인덱싱

시간 열을 **인덱스**로 끌어올리는 순간 pandas의 모든 시간 기능이 열립니다. 열로 두면 평범한 열이지만, 인덱스가 되면 시계열로 격상됩니다.

3-1. 실습 3 — set_index와 sort_index

메서드 .set_index() 특정 열을 인덱스 이름표로 끌어올리기

정의 인덱스로 삼을 열 이름을 넣습니다. 원본을 바꾸지 않고 새 객체를 돌려주므로 반드시 다시 담아야 합니다.

형태 df = df.set_index("timestamp")

```
df_3 = pd.read_csv("Data/20_engine01_timestamp_sample.csv")
df_3["timestamp"] = pd.to_datetime(df_3["timestamp"])
```

```
# 인덱스로 올리고 정렬 — 결과를 반드시 다시 담기
df_3 = df_3.set_index("timestamp").sort_index()
```

```
print("인덱스 타입:", df_3.index.dtype)
print("인덱스 클래스:", type(df_3.index).__name__)
print("첫 시각 :", df_3.index[0])
print("마지막 시각:", df_3.index[-1])
print("전체 행 :", len(df_3))
```

- ▶ 인덱스 타입: datetime64[ns]
- ▶ 인덱스 클래스: DatetimeIndex
- ▶ 첫 시각 : 2024-03-01 00:00:00
- ▶ 마지막 시각: 2024-03-08 23:00:00
- ▶ 전체 행 : 182

왜 쓰나 DatetimeIndex가 되어야 .loc에 시각을 넣거나 resample()을 쓸 수 있습니다. 인덱스가 시각이 되는 순간 pandas가 이 표를 시계열로 인식합니다.

응용 .set_index()와 .sort_index()는 둘 다 새 객체를 돌려주므로 **점으로 이어 쓸 수 있습니다**. 실습 코드가 한 줄로 붙여 쓴 이유입니다.

```
# 세 단계를 한 줄로 (메서드 체이닝)
df_3 = (pd.read_csv("Data/20_engine01_timestamp_sample.csv")
        .assign(timestamp=lambda x: pd.to_datetime(x["timestamp"]))
        .set_index("timestamp")
        .sort_index())

# 다시 열로 되돌리기
back = df_3.reset_index()
print(back.columns.tolist()[:2])

▶ ['timestamp', 'temp_out']
```

개념

df =를 빠뜨리는 것이 가장 흔한 실수입니다. df.set_index("timestamp")라고만 쓰면 화면에는 결과가 보이지만 df는 그대로입니다. 다음 줄에서 resample()을 쓰면 "인덱스가 시간이 아니다"라는 오류가 나는데, 원인을 찾기 어렵습니다.

주의

inplace=True를 쓰면 다시 담지 않아도 되지만 **pandas 공식적으로 권장하지 않는 방식**입니다. 메서드 체이닝이 안 되고, 향후 제거될 가능성도 있습니다. **결과를 다시 담는 습관**을 들이는 편이 낫습니다.

메서드 .sort_index() 인덱스 기준 오름차순 정렬

정의 인덱스를 기준으로 행 순서를 맞춥니다. 시간 인덱스라면 **시각 오름차순**이 됩니다.

형태 df = df.sort_index()

```
# 정렬 전후 확인
print("정렬돼 있나?:", df_3.index.is_monotonic_increasing)

▶ 정렬돼 있나?: True
```

왜 쓰나 pandas는 기간 자르기가 정렬을 전제로 동작합니다. 정렬이 안 돼 있으면 .loc["2024-03-05": "2024-03-07"]이 오류를 내거나 빈 결과를 돌려줍니다. "시작과 끝 사이"라는 개념 자체가 성립하지 않기 때문입니다.

응용 여러 CSV를 concat으로 합치면 **시간 순서가 어긋나는 일이 흔합니다**. 파일마다 기간이 겹치거나 순서가 뒤바뀌기 때문입니다.

```
# 파일을 합친 뒤에는 반드시 정렬
# df = pd.concat([df_a, df_b, df_c])
# df = df.sort_index()

# 정렬 여부는 이 속성으로 확인
print(df_3.index.is_monotonic_increasing)
```

개념

`is_monotonic_increasing`은 **괄호가 없는 속성**입니다. 인덱스가 계속 커지기만 하는지를 True/False로 알려줍니다. 시계열 작업을 시작하기 전에 한 번 짚어 보면 문제를 미리 잡을 수 있습니다.

주의

정렬은 **한 번만 해 두면 이후 모든 분석에서 효과가 유지**됩니다. 매번 할 필요가 없으므로, 데이터를 읽는 셀에서 한꺼번에 처리하는 것이 좋습니다.

3-2. 실습 4 — 부분 문자열 인덱싱

시간 인덱스의 진짜 위력은 여기서 나옵니다. **시각의 일부만 적으면 그 기간 전체**가 선택됩니다.

적는 법	범위	예	결과
"2024-03-01 09:00:00"	한 행	한 시점	행 하나
"2024-03-02"	그날 전체	하루	20행
"2024-03"	그달 전체	한 달	182행 (전부)
"2024-03-05" : "2024-03-07"	기간 전체	사흘	70행 (끝 포함)

Practice/20_02_example.ipynb — 실습 4

```
df_4 = pd.read_csv("Data/20_engine01_timestamp_sample.csv")
df_4["timestamp"] = pd.to_datetime(df_4["timestamp"])
df_4 = df_4.set_index("timestamp").sort_index()
```

```
# 하루 날짜를 넣어 그날 데이터만
```

```
day_4 = df_4.loc["2024-03-02"]
```

```
print("2024-03-02 하루:", len(day_4), "행")
```

```
# 시작과 끝을 콜론으로 이어 기간 선택 — 끝 날짜를 포함합니다
```

```
period_4 = df_4.loc["2024-03-05":"2024-03-07"]
```

```
print("2024-03-05 ~ 03-07:", len(period_4), "행")
```

```
print("기간 끝 시각:", period_4.index[-1])
```

▶ 2024-03-02 하루: 20 행

▶ 2024-03-05 ~ 03-07: 70 행

▶ 기간 끝 시각: 2024-03-07 23:00:00

주의

시간 인덱스 슬라이싱은 끝을 포함합니다. 파이썬 리스트에서 `lst[0:3]`이 세 개만 가져오는 것과 정반대입니다.

"2024-03-05":"2024-03-07"이라고 쓰면 **7일도 들어갑니다.** "고장 직전 일주일"처럼 사람이 말하는 방식과 같아 오히려 직관적이지만, 리스트 습관 때문에 헷갈리기 쉽습니다.

3-3. 하루가 24행이 아닌 이유

여기서 **오늘 데이터의 숨은 문제**가 드러납니다. 하루면 24행이어야 하는데 3월 2일은 **20행**입니다. 사흘이면 72행이어야 하는데 **70행**입니다.

센서 로그는 시각이 통째로 빠질 수 있습니다

이론상 있어야 할 시각을 만들어 실제와 비교

```
전체시각_4 = pd.date_range(df_4.index[0], df_4.index[-1], freq="1h")
```

```
print("이론상 시간 수:", len(전체시각_4))
```

```
print("실제 행 수 :", len(df_4))
```

```
print("빠진 시각 :", len(전체시각_4) - len(df_4), "개")
```

```
print()
```

```
print(전체시각_4.difference(df_4.index))
```

▶ 이론상 시간 수: 192

▶ 실제 행 수 : 182

▶ 빠진 시각 : 10 개

```
▶ DatetimeIndex(['2024-03-01 10:00:00', '2024-03-01 11:00:00',  
▶                '2024-03-02 01:00:00', '2024-03-02 16:00:00',  
▶                '2024-03-02 17:00:00', '2024-03-02 18:00:00',  
▶                '2024-03-04 05:00:00', '2024-03-04 18:00:00',  
▶                '2024-03-06 00:00:00', '2024-03-06 01:00:00'],  
▶                dtype='datetime64[ns]', freq=None)
```

함수 `pd.date_range()` 일정 간격의 시각 목록 만들기

정의 시작과 끝, 간격을 주면 **그 사이의 모든 시각**을 만들어 줍니다. "있어야 할 시각"의 정답지 역할을 합니다.

형태 `pd.date_range(시작, 끝, freq="1h")`

시작·끝·간격으로 만들기

```
r = pd.date_range("2024-03-01", "2024-03-02", freq="6h")  
print(r)
```

개수로 만들기

```
r2 = pd.date_range("2024-03-01", periods=4, freq="1D")  
print(r2)
```

```
▶ DatetimeIndex(['2024-03-01 00:00:00', '2024-03-01 06:00:00',  
▶                '2024-03-01 12:00:00', '2024-03-01 18:00:00',  
▶                '2024-03-02 00:00:00'], dtype='datetime64[ns]', freq='6h')  
▶ DatetimeIndex(['2024-03-01', '2024-03-02', '2024-03-03', '2024-03-04'],  
▶                dtype='datetime64[ns]', freq='D')
```

왜 쓰나 누락된 시각을 찾아내는 데 씁니다. 실제 데이터와 정답지를 비교하면 어느 시각이 빠졌는지 정확히 알 수 있습니다. 로그가 끊긴 시점을 찾는 실무 기법입니다.

응용 `.difference()`는 인덱스끼리의 **차집합**입니다. 정답지에는 있는데 실제에는 없는 시각만 남깁니다.

```
# 누락 시각이 어느 날에 몰려 있는지 세어 보기
빠짐 = 전체시각_4.difference(df_4.index)
print(빠짐.to_series().dt.date.value_counts().sort_index())
```

```
# 실제 일별 행 수
print(df_4.resample("1D").size())
```

```
▶ 2024-03-01    2
▶ 2024-03-02    4
▶ 2024-03-04    2
▶ 2024-03-06    2
▶
▶ 2024-03-01    22
▶ 2024-03-02    20
▶ 2024-03-03    24
▶ 2024-03-04    22
▶ 2024-03-05    24
▶ 2024-03-06    22
▶ 2024-03-07    24
▶ 2024-03-08    24
```

개념

8일 중 4일에서 시각이 빠졌습니다. 3월 2일이 4개로 가장 많이 빠졌고, 3·5·7·8일은 온전한 24행입니다.

`freq="1h"`의 `h`는 **소문자**입니다. 예전에는 대문자 `H`였지만 지금은 소문자가 표준이고, 대문자를 쓰면 경고가 뜹니다.

설비 적용

센서 로그에서 시각이 통째로 빠지는 것은 흔한 일입니다. 통신 두절, 설비 정지, 로거 재시작 등이 원인입니다.

문제는 **행 개수를 당연하게 믿으면 안 된다**는 점입니다. "하루 24행"을 전제로 계산하면 전부 어긋납니다. 그리고 STEP 4에서 보듯 `resample`은 이 빈 시각을 **자동으로 채워 넣습니다.**

STEP 4

리샘플링으로 묶기

resample · 빈도 문자열 · 집계 함수

시간 간격을 바꾸는 도구입니다. 1시간마다 잦은 값을 하루 단위 평균으로 묶으면 잡음이 사라지고 흐름만 남습니다.

4-1. 다운샘플링과 업샘플링

구분		
방향	큰 단위로 묶기	작은 단위로 쪼개기
예	1시간 → 1일	1시간 → 30분
데이터 양	줄어듦	늘어남
결측	발생하지 않음	반드시 생김
효과	잡음이 평균으로 사라짐	채우는 방식을 따로 정해야 함
설비 분석에서	훨씬 자주 씀	드물

비유

매일 잦은 몸무게를 주 단위 평균으로 보면 흐름이 훨씬 잘 보이는 것과 같습니다. 하루하루는 물 마신 양이나 식사 시간에 따라 오르내리지만, 주 단위로 묶으면 진짜 변화만 남습니다. 설비 데이터도 마찬가지로 다운샘플링이 압도적으로 자주 쓰입니다.

4-2. 빈도 문자열

글자	단위	예	대소문자
min	분	resample("15min")	소문자
h	시간	resample("6h")	소문자
D	일	resample("1D")	대문자
W	주	resample("W")	대문자
ME	월말	resample("ME")	대문자

주의

시간보다 작은 단위는 소문자, 날짜 이상은 대문자입니다. 이 규칙 하나만 기억하면 됩니다.

그리고 표기가 바뀌었습니다. 예전에는 시간이 대문자 H, 분이 대문자 T였는데 지금은 소문자 h와 min이 표준입니다. 인터넷 예제에 "H"나 "T"가 보이면 "h"와 "min"으로 바꿔 쓰십시오. 아직 동작은 하지만 FutureWarning이 뜹니다.

4-3. 실습 5 — resample과 집계 함수

메서드 `.resample()` 시간 단위를 다시 묶기

정의 시간 인덱스가 있어야만 동작합니다. 그리고 묶기만 해서는 아무 일도 안 일어나므로 집계 함수를 반드시 뒤에 붙여야 합니다.

형태 `df["열"].resample("6h").mean()`

```
df_5 = pd.read_csv("Data/20_engine01_timestamp_sample.csv")
df_5["timestamp"] = pd.to_datetime(df_5["timestamp"])
df_5 = df_5.set_index("timestamp").sort_index()

# 6시간 단위 평균
print(df_5["temp_out"].resample("6h").mean().head(3).round(3))

# 같은 단위로 최댓값
print(df_5["temp_out"].resample("6h").max().head(3))
```

▶ timestamp	
▶ 2024-03-01 00:00:00	641.058
▶ 2024-03-01 06:00:00	641.005
▶ 2024-03-01 12:00:00	640.953
▶	
▶ 2024-03-01 00:00:00	641.18
▶ 2024-03-01 06:00:00	641.27
▶ 2024-03-01 12:00:00	641.23

왜 쓰나 설비 분석에서는 보통 평균으로 흐름을 보고 최댓값으로 위험한 순간을 점검합니다. 평균만 보면 순간적으로 치솟은 값이 묻혀 사라지기 때문입니다.

응용 여러 집계를 한 번에 보려면 `agg()`를 씁니다. 24일차에 배운 그 메서드가 여기서도 그대로 쓰입니다.

```
# 6시간 단위 종합
print(df_5["temp_out"].resample("6h")
      .agg(["mean", "max", "min", "count"]).head(3).round(3))
```

```

>
> timestamp
> 2024-03-01 00:00:00    641.058    641.18    640.66         6
> 2024-03-01 06:00:00    641.005    641.27    640.84         4
> 2024-03-01 12:00:00    640.953    641.23    640.71         6

```

개념

count 열에 주목하십시오. 첫 구간은 **6개**인데 두 번째 구간은 **4개**입니다. 3월 1일 10시와 11시가 빠졌기 때문입니다. count를 함께 보면 어느 구간에 데이터가 부족한지 바로 알 수 있습니다. **평균이 4개로 계산된 것과 6개로 계산된 것은 신뢰도가 다릅니다.**

주의

resample()은 **뭉치만 하는 중간 단계**입니다. 뒤에 집계 함수를 안 붙이면 Resampler 객체가 그대로 남아 아무 값도 나오지 않습니다. `.rolling()`도 똑같은 구조라, 두 메서드는 **뒤에 뭔가를 붙여야 완성된다고** 기억하십시오.

4-4. 실습 6 — 단위를 바꾸면 행 수가 달라집니다

Practice/20_02_example.ipynb — 실습 6

```
print("원본 행 수:", len(df_6))
print()
for unit in ["1h", "6h", "12h", "1D"]:
    결과 = df_6["temp_out"].resample(unit).mean()
    print(f"{unit:4} -> {len(결과):4}행")
```

```

> 원본 행 수: 182
>
> 1h    ->   192행
> 6h    ->   32행
> 12h   ->   16행
> 1D    ->    8행

```

자주 하는 실수

1h로 묶었더니 182행이 192행으로 오히려 늘었습니다. 원본이 이미 1시간 간격인데 왜 늘었을까요.

resample이 빠진 10개 시각을 NaN 자리로 채워 넣기 때문입니다. STEP 3에서 찾아낸 그 10개입니다. 즉

resample("1h")은 다운샘플링도 업샘플링도 아니고 **누락 시점을 드러내는 재배치**가 됩니다.

이것을 모르고 "리샘플링하면 항상 행이 준다"고 외우면, 결과가 늘어났을 때 코드가 잘못된 줄 알고 엉뚱한 곳을 찾게 됩니다.

정확히 누락 시각 개수와 일치합니다

늘어난 10행이 전부 NaN인지 확인

```
print("1h 리샘플 후 NaN 개수:", df_6["temp_out"].resample("1h").mean().isna().sum())
```

▶ 1h 리샘플 후 NaN 개수: 10

메서드 .asfreq() 집계 없이 간격만 다시 맞추기

정의 resample이 묶어서 요약하는 도구라면, asfreq는 해당 시점의 값만 그대로 가져오는 도구입니다.

형태 df["열"].asfreq("1h")

누락 시점을 드러내는 용도

```
print("asfreq 결측:", df_6["temp_out"].asfreq("1h").isna().sum())
```

resample과 비교 — 같은 10개

```
print("resample 결측:", df_6["temp_out"].resample("1h").mean().isna().sum())
```

▶ asfreq 결측: 10

▶ resample 결측: 10

왜 쓰나 집계 함수를 붙일 필요가 없습니다. 누락 시점이 몇 개인지만 빠르게 확인할 때 resample().mean()보다 의도가 분명합니다.

응용 두 도구의 차이는 한 구간에 값이 여러 개일 때 드러납니다.

6시간으로 묶으면 차이가 보입니다

```
print(df_6["temp_out"].resample("6h").mean().head(2).round(3)) # 6개를 평균
print(df_6["temp_out"].asfreq("6h").head(2)) # 그 시점 값만
```

```
▶ 2024-03-01 00:00:00    641.058
▶ 2024-03-01 06:00:00    641.005
▶
▶ 2024-03-01 00:00:00    641.17
▶ 2024-03-01 06:00:00    641.01
```

개념

`resample("6h").mean()`은 00시~05시 여섯 값의 평균 **641.058**을 주고, `asfreq("6h")`는 00시 **정각의 값 641.17**만 줍니다. 완전히 다른 숫자입니다. 평소 집계는 **resample**, 누락 점검은 **asfreq**로 나눠 쓰십시오.

자주 하는 실수

`asfreq`를 집계 목적으로 쓰면 데이터를 대부분 버리게 됩니다. 6시간마다 한 값만 남기므로 나머지 다섯 값의 정보가 통째로 사라집니다. 이름이 비슷해 헷갈리기 쉬운 함정입니다.

4-5. 어느 단위로 묶을 것인가

집계 단위는 보고 싶은 변화의 시간 규모에 맞춥니다. 사진을 가까이서 찍느냐 멀리서 찍느냐와 같은 선택입니다.

보고 싶은 것	단위	이유	오늘 데이터
순간 이상	min	짧은 스파이크·이상 신호 포착	원본이 1시간이라 불가
하루 내 변화	h	24시간 안의 운전 패턴	192행 — 원본과 같음
며칠 추세	D	일별 평균이 추세를 또렷하게	8행 — 가장 잘 보임
장기 추세	W	주 단위로 큰 그림	8일뿐이라 점이 두 개

개념

너무 작으면 잡음이 그대로 남아 흐름이 안 보이고, 너무 크면 의미 있는 변화가 평균에 묻혀 사라집니다. 한 달치를 분 단위로 묶으면 흐름이 안 보이고, 월 단위로 묶으면 점이 하나뿐입니다.

한 단위만 고집하지 마십시오. 큰 단위로 전체를 잡고 작은 단위로 관심 구간을 확대하는 것이 실무 방식입니다. 지도에서 전체를 보다 관심 지역을 확대하는 것과 같습니다.

집계의 효과	무엇	주의
① 잡음 제거	잔잔한 흔들림이 평균으로 상쇄	너무 크게 묶으면 위험 스파이크도 묻힘
② 시간대 비교	시간대·요일별 운영 패턴 분석	timestamp만의 특권 — cycle로는 불가
③ 데이터 경량화	행 수가 줄어 다루기 쉬워짐	큰 흐름은 보존됨

주의

너무 큰 단위로 묶으면 위험한 스파이크가 평균에 묻힙니다. 하루 평균이 641도로 정상이어도 그날 한때 700도를 찍었을 수 있습니다. 그래서 실무에서는 **평균과 최댓값을 함께 봅니다**. `agg(["mean", "max"])` 한 줄이면 둘 다 나옵니다.

STEP 5

추세 확인과 업샘플링

일별 평균 · 결측 발생 · interpolate

리샘플링을 배웠으니 이제 **실제로 무엇을 얻는지**를 봅시다. 잡음에 가려 안 보이던 추세가 일별 평균에서는 또렷하게 드러납니다.

5-1. 실습 7 — 일별 집계로 추세 확인

Practice/20_02_example.ipynb — 실습 7

```
df_7 = pd.read_csv("Data/20_engine01_timestamp_sample.csv")
df_7["timestamp"] = pd.to_datetime(df_7["timestamp"])
df_7 = df_7.set_index("timestamp").sort_index()
```

하루 단위 평균

```
daily_7 = df_7["temp_out"].resample("1D").mean()
print(daily_7.round(2))
```

```
▶ timestamp
▶ 2024-03-01    641.07
▶ 2024-03-02    641.34
▶ 2024-03-03    641.56
▶ 2024-03-04    641.68
▶ 2024-03-05    641.84
▶ 2024-03-06    642.10
▶ 2024-03-07    642.20
▶ 2024-03-08    642.35
▶ Freq: D, Name: temp_out, dtype: float64
```

출렁이는 원본 사이로 매끄러운 추세선이 지나갑니다

```
# 원본을 열게, 일별 평균을 진하게 겹쳐 그리기
plt.figure(figsize=(12, 5))
plt.title("temp_out - 시간별 원본과 일별 평균 추세")
plt.plot(df_7.index, df_7["temp_out"], alpha=0.4, label="시간별 원본")
plt.plot(daily_7.index, daily_7.values, linewidth=3, color="red", label="일별 평균")
plt.legend()
plt.show()

# 첫날과 마지막날 비교
첫날_7, 끝날_7 = daily_7.iloc[0], daily_7.iloc[-1]
방향_7 = "상승" if 끝날_7 > 첫째날_7 else "하강"
print(f"첫날 {첫날_7:.2f} -> 마지막날 {끝날_7:.2f}"
      f" ({방향_7}, 변화량 {끝날_7 - 첫째날_7:+.2f})")

# 두 점만 보지 말고 8일 내내 단조 증가인지 확인
print("매일 전날보다 높았는가?:", (daily_7.diff().dropna() > 0).all())
```

▶ 첫째날 641.07 -> 마지막날 642.35 (상승, 변화량 +1.29)
▶ 매일 전날보다 높았는가?: True

설비 적용

시간별 원본은 위아래로 튀어서 방향이 안 보이지만, 일별 평균은 641.07에서 642.35로 8일 내내 한 번도 안 꺾이고 올랐습니다. 이것이 다운샘플링의 목적입니다.

그리고 `.diff()`로 확인한 "매일 전날보다 높았는가"가 True라는 점이 중요합니다. 첫째날과 끝날 두 점만 비교하면 중간에 오르내렸을 가능성을 배제할 수 없는데, **8일 연속 단조 증가**라면 우연이 아닙니다.

변화량은 겨우 **1.29°C**입니다. 세로축을 0~1000으로 잡으면 완전히 평평해 보였을 값인데, 어제 배운 대로 **축 범위를 좁혀야 보이는 크기**입니다.

메서드 `.diff()` 직전 값과의 차이 구하기

정의 각 값에서 바로 앞 값을 뺀 결과를 돌려줍니다. 첫 값은 뺄 대상이 없어 NaN이 됩니다.

형태 `series.diff()`

```
# 일별 평균의 전날 대비 변화량
print(daily_7.diff().round(3))
```

```
▶ timestamp
▶ 2024-03-01      NaN
▶ 2024-03-02      0.273
▶ 2024-03-03      0.218
▶ 2024-03-04      0.117
▶ 2024-03-05      0.168
▶ 2024-03-06      0.252
▶ 2024-03-07      0.108
▶ 2024-03-08      0.151
▶ Freq: D, Name: temp_out, dtype: float64
```

왜 쓰나 "오르고 있다"를 눈이 아니라 숫자로 증명할 수 있습니다. 전부 양수면 단조 증가입니다. 그래프를 눈으로 보고 판단하는 것보다 훨씬 객관적입니다.

응용 다음 시간에 배울 **변화량 분석의 기본 도구**입니다. 급변 지점을 찾을 때도 이 메서드를 씁니다.

```
# 변화가 가장 컸던 날
d = daily_7.diff()
print("가장 많이 오른 날:", d.idxmax().date(), round(d.max(), 3))
print("가장 적게 오른 날:", d.idxmin().date(), round(d.min(), 3))
```

```
▶ 가장 많이 오른 날: 2024-03-02 0.273
▶ 가장 적게 오른 날: 2024-03-07 0.108
```

개념

`.diff()`는 **NaN이 하나 생기므로** 뒤에 `.dropna()`를 붙이거나 조건 검사 시 주의해야 합니다.
`(daily_7.diff() > 0).all()`이라고 쓰면 NaN이 False로 취급되어 결과가 False가 나옵니다. 실습 코드가 `.dropna()`를 붙인 이유입니다.

주의

`.diff(periods=2)`처럼 쓰면 **두 칸 앞과** 비교합니다. 하루 전이 아니라 일주일 전과 비교하고 싶을 때 유용합니다.

5-2. 실습 8 — 업샘플링과 보간

반대 방향입니다. **1시간 간격을 30분으로 쪼개면** 없던 시점이 생기고 그 자리는 전부 결측이 됩니다.

```
# 30분 단위로 촘촘하게
up_8 = df_8["flow"].resample("30min").mean()
```

```
print("업샘플링 후 전체 길이:", len(up_8))
print("업샘플링 결측:", up_8.isna().sum())
```

```
# 보간으로 빈 값 채우기
채움_8 = up_8.interpolate()
print("보간 후 결측:", 채움_8.isna().sum())
```

```
▶ 업샘플링 후 전체 길이: 383
▶ 업샘플링 결측: 201
▶ 보간 후 결측: 0
```

개념

원본 182행이 30분 단위로 쪼개지며 **383행**이 되었습니다. 그중 **201개가 결측**입니다. 원래 없던 30분 자리 181개에 더해, STEP 3에서 확인한 **누락 시각 10개가 정각과 30분 두 자리씩 20개로** 늘어난 결과입니다. **업샘플링은 결측을 만드는 작업**이라는 것을 숫자로 확인한 셈입니다.

메서드 `.interpolate()` 앞뒤 값을 이어 중간값으로 채우기

정의 앞뒤 값을 잇는 직선 위에서 중간값을 계산해 빈 자리를 메웁니다. 시계열에 가장 자연스러운 채움 방식입니다.

형태 `series.interpolate()`

```
# 세 가지 채움 방식 비교
s = pd.Series([10, None, None, 40])

print("ffill      :", s.fffll().tolist())
print("bfill      :", s.bfill().tolist())
print("interpolate:", s.interpolate().tolist())
```

```
▶ ffill      : [10.0, 10.0, 10.0, 40.0]
▶ bfill      : [10.0, 40.0, 40.0, 40.0]
▶ interpolate: [10.0, 20.0, 30.0, 40.0]
```

왜 쓰나 `ffill`은 앞 값을 그대로 복사하므로 **계단 모양**이 되고, `interpolate`는 **매끄러운 직선**이 됩니다. 온도처럼 서서히 변하는 값에는 보간이 훨씬 자연스럽습니다.

응용 23일차에 배운 `fillna` 계열과 같은 자리에 있지만, **시계열 전용**이라는 점이 다릅니다. 원본의 기존 결측에도 그대로 쓸 수 있습니다.

```
# 원본에 원래 있던 결측도 보간으로
print(df_8.isna().sum().to_dict())

for col in ["vibration", "pressure"]:
    전 = df_8[col].isna().sum()
    후 = df_8[col].interpolate().isna().sum()
    print(f"{col:10} 보간 전 {전}개 -> 보간 후 {후}개")
```

```
> {'temp_out': 0, 'temp_core': 0, 'pressure': 2, 'vibration': 3, 'flow': 0}
> vibration   보간 전 3개 -> 보간 후 0개
> pressure    보간 전 2개 -> 보간 후 0개
```

개념

보간은 추정입니다. 실제로 측정한 값이 아니라 앞뒤로 미루어 만든 값입니다. 너무 많은 구간을 채우면 실제와 멀어지므로, **보간한 값과 실제 측정값은 구분해서 다루는 편이 좋습니다.** 예를 들어 보간 전에 결측 위치를 따로 기록해 두는 식입니다.

자주 하는 실수

맨 앞이나 맨 뒤가 결측이면 `interpolate()`가 채우지 못합니다. **앞뒤 중 한쪽이 없어 직선을 그릴 수 없기 때문입니다.** 이때는 `limit_direction="both"`를 주거나 `ffill.bfill`을 함께 써야 합니다.

채움 방식	원리	적합한 값	오늘 데이터
<code>.ffill()</code>	바로 앞 값을 그대로	상태·설정처럼 유지되는 값	—
<code>.bfill()</code>	바로 뒤 값을 그대로	뒤에서 앞으로 채울 때	—
<code>.interpolate()</code>	앞뒤 사이 직선의 중간값	온도처럼 서서히 변하는 값	201 → 0

연결

23일차에 `fillna(0)`이나 평균 대체가 **분산을 줄인다고** 배웠습니다. 보간도 같은 성격의 부작용이 있습니다. 앞뒤를 이은 직선 위의 값이라 **원래 있었을 흔들림이 없어집니다.** 그래서 보간을 많이 한 데이터로 STEP 7의 이동표준편차를 재면 **변동성이 실제보다 작게** 나옵니다. 순서를 바꿔 생각하면, **변동성을 볼 목적이라면 보간을 신중하게** 해야 한다는 뜻입니다.

STEP 6

노이즈와 이동평균

rolling().mean() · 윈도우 크기

다시 cycle 데이터로 돌아옵니다. 여기서부터가 **21_01 추세와 변동성**입니다. 데이터는 21_cmapss_fd001_sample.csv로 바꿉니다.

6-1. 노이즈와 추세

용어	뜻	설비에서
노이즈	센서값이 작게 위아래로 떨리는 현상	측정 오차 · 순간 조건 변화 · 외부 요인
추세	자잘한 떨림을 무시했을 때의 전체 방향	설비의 장기 건강 상태
평활화	노이즈를 줄여 데이터를 부드럽게 다듬기	이동평균이 대표 기법

개념

노이즈 없는 완벽히 매끈한 센서 데이터는 현실에 없습니다. 그래서 목표는 노이즈를 없애는 것이 아니라 **노이즈에 가려진 진짜 변화를 찾는 것**입니다.

그리고 완만한 추세는 노이즈에 가려 **눈으로 읽기 어렵습니다**. 같은 그래프를 보고도 사람마다 다르게 판단합니다.

"그래프 보고 알아서 판단"은 분석이 아닙니다. 누가 봐도 같은 결론이 나오는 객관적 계산이 필요합니다.

6-2. 실습 1 — 센서 시계열 그려 보기

Practice/21_01_example.ipynb — 실습 1

```
df_1 = pd.read_csv("Data/21_cmapss_fd001_sample.csv")
print("shape:", df_1.shape)
print(df_1["unit_nr"].value_counts().sort_index())

# 2번 엔진을 골라 회차 순으로 정렬
engine_1 = df_1[df_1["unit_nr"] == 2].sort_values("time_cycles")
print("2번 엔진:", engine_1.shape)

plt.figure(figsize=(10, 4))
plt.title("2번 엔진 - s_4 센서 추세")
plt.plot(engine_1["time_cycles"], engine_1["s_4"])
plt.xlabel("time_cycles")
plt.ylabel("s_4")
```

```
plt.show()

# 눈으로 본 방향을 숫자로 계산
첫값_1 = engine_1["s_4"].iloc[0]
끝값_1 = engine_1["s_4"].iloc[-1]
r_1 = engine_1["time_cycles"].corr(engine_1["s_4"])
print(f"첫값 {첫값_1:.2f} -> 끝값 {끝값_1:.2f}"
      f" (변화량 {끝값_1 - 첫값_1:+.2f}, 회차 상관 r={r_1:+.3f})")
```

```
> shape: (1031, 10)
> unit_nr
> 1      192
> 2      287
> 3      179
> 4      218
> 5      155
> 2번 엔진: (287, 10)
>
> 첫값 1399.71 -> 끝값 1416.77 (변화량 +17.06, 회차 상관 r=+0.989)
```

설비 적용

상관계수 +0.989는 거의 직선입니다. 점 하나하나의 위아래로 튀지만 전체 흐름은 확실한 우상향입니다. 그리고 이 엔진은 **287회차에서 고장**났습니다. s_4가 1399.71에서 1416.77로 17.06만큼 올라간 것이 **열화(degradation)의 흔적**입니다. 엔진 수명이 155~287회차로 제각각인 것과 함께 보면, **같은 모델이라도 개체마다 다르게 닳는**다는 것을 알 수 있습니다.

6-3. 실습 2 — 이동평균 구하기

메서드 `.rolling()` 창을 미끄러뜨리며 구간 계산

정의 window만큼의 값을 한 묶음으로 보는 **창(window)**을 **한 칸씩 밀며** 계산합니다. 뒤에 `.mean()`이나 `.std()`를 붙여야 결과가 나옵니다.

형태 `series.rolling(window=10).mean()`

```
# 교안 예시로 원리 확인
s = pd.Series([10, 12, 15, 14, 18])
print(s.rolling(window=3).mean().tolist())

> [nan, nan, 12.33, 13.67, 15.67]
```

왜 쓰나 $(10+12+15)/3 = 12.33$, $(12+15+14)/3 = 13.67$, $(15+14+18)/3 = 15.67$ 입니다. 앞 두 개는 묶음 값이 부족해 NaN이 됩니다. 이것이 정상 동작입니다.

응용 실제 데이터에 적용하면 **원본의 흔들림이 줄어든 것**이 숫자로 확인됩니다.

```
engine_2 = df_2[df_2["unit_nr"] == 2].sort_values("time_cycles")
ma10_2 = engine_2["s_4"].rolling(window=10).mean()
```

```
print("이동평균 NaN 개수:", ma10_2.isna().sum(), "(= window - 1)")
print("원본 표준편차 :", round(engine_2["s_4"].std(), 3))
print("이동평균 표준편차 :", round(ma10_2.std(), 3))
```

▶ 이동평균 NaN 개수: 9 (= window - 1)

▶ 원본 표준편차 : 5.084

▶ 이동평균 표준편차 : 4.875

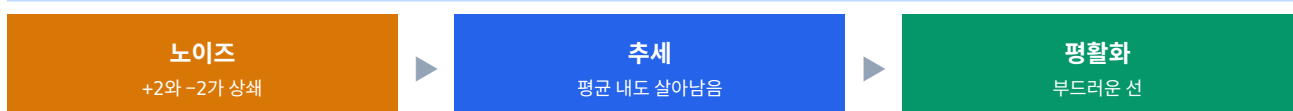
개념

NaN 개수는 항상 window - 1개입니다. 창 크기가 10이면 9개, 20이면 19개입니다. 이 규칙을 알면 앞부분이 비는 것이 오류가 아님을 바로 알 수 있습니다.

설비 적용

원본 표준편차 5.084가 이동평균에서 4.875로 줄었습니다. 잡음이 걸러진 만큼 흔들림이 작아진 것입니다. 다만 4% 정도밖에 안 줄었는데, s_4는 애초에 노이즈보다 추세가 훨씬 큰 센서이기 때문입니다. 표준편차 5.084 중 대부분이 노이즈가 아니라 1399→1417이라는 상승분입니다.

6-4. 이동평균이 추세를 드러내는 원리



성분	원본	평균 후	결과
노이즈	100 · 102 · 98	평균 100	사라짐
추세	100 · 101 · 102	평균 101	살아남음

개념

한 문장으로 요약하면 "노이즈는 상쇄되어 사라지고, 추세는 살아남는다"입니다. 노이즈는 위아래로 무작위로 튀므로 여러 개를 평균 내면 +와 -가 만나 0에 가까워집니다. 반면 추세는 한 방향으로만 움직이므로 평균을 내도 그 방향이 그대로 남습니다. 이것이 이동평균이 작동하는 전체 원리입니다.

6-5. 실습 3 — 윈도우 크기의 맞교환

Practice/21_01_example.ipynb — 실습 3

```
for w in [5, 20, 50]:
    ma_3 = engine_3["s_4"].rolling(window=w).mean()
    첫회차_3 = engine_3.loc[ma_3.first_valid_index(), "time_cycles"]
    print(f"w={w:3} | 앞쪽 NaN {ma_3.isna().sum():3}개"
          f" | 표준편차 {ma_3.std():.3f}"
          f" | 값이 처음 나오는 회차 {첫회차_3}")
```

- ▶ w= 5 | 앞쪽 NaN 4개 | 표준편차 4.968 | 값이 처음 나오는 회차 5
- ▶ w= 20 | 앞쪽 NaN 19개 | 표준편차 4.703 | 값이 처음 나오는 회차 20
- ▶ w= 50 | 앞쪽 NaN 49개 | 표준편차 4.194 | 값이 처음 나오는 회차 50

구분		
선 모양	거칠게 떨림	매끈함
표준편차	4.968	4.194
변화 반응	빠름	느림
앞쪽 빈 구간	4회차	49회차
287회차 중 손실	1.4%	17.1%

개념

큰 윈도우가 무조건 좋은 것이 아닙니다. 매끈해지는 대신 둔감해지고 앞부분이 많이 빕니다. w=50이면 287회차 중 앞 49회차를 통째로 못 쓰게 됩니다.

교안이 준 기준은 192주기 데이터에 윈도우 10~30입니다. 정답은 없고 분석 목적에 맞는 안경 도수를 고르는 것과 같습니다. 높은 도수 안경은 흐릿하지만 큰 윤곽이 보이고, 낮은 도수는 선명하지만 어수선향니다.

메서드 `.first_valid_index()` 첫 유효값의 인덱스 라벨 찾기

정의 NaN이 아닌 첫 값의 인덱스 라벨을 돌려줍니다. 값 자체가 아니라 위치를 얻는다는 점이 중요합니다.

형태 `series.first_valid_index()`

```
ma = engine_3["s_4"].rolling(window=20).mean()

print("첫 유효 인덱스 라벨:", ma.first_valid_index())
print("그 자리의 회차 :", engine_3.loc[ma.first_valid_index(), "time_cycles"])
print("그 자리의 값 :", round(ma.loc[ma.first_valid_index()], 3))
```

```
▶ 첫 유효 인덱스 라벨: 211
▶ 그 자리의 회차 : 20
▶ 그 자리의 값 : 1400.67
```

왜 쓰나 라벨이 211인데 회차는 20입니다. 걸러 낸 데이터라 인덱스가 0부터 시작하지 않기 때문입니다. 회차를 알고 싶으면 `.loc`으로 `time_cycles` 열에서 다시 찾아야 합니다.

응용 `.last_valid_index()`도 있고, `.dropna()`로 아예 NaN을 버리는 방법도 있습니다.

```
# NaN을 버리고 첫 값만
print(round(ma.dropna().iloc[0], 3))

# 유효 구간의 길이
print("유효 구간:", ma.notna().sum(), "/", len(ma))
```

```
▶ 1400.67
▶ 유효 구간: 268 / 287
```

개념

287 - 19 = 268입니다. 윈도우 20이면 앞 19개가 비므로 유효 구간이 268회차입니다. STEP 7의 $\pm 2\sigma$ 밴드에서 이 268이라는 숫자가 다시 나옵니다.

자주 하는 실수

`.loc[]`에 넘길 때 **위치 번호와 라벨을 헷갈리면 조용히 엉뚱한 행이 선택됩니다.** `first_valid_index()`는 라벨을 주므로 반드시 `.loc`과 짝지어야 하고, `.iloc`과 함께 쓰면 안 됩니다.

6-6. 실습 4 — 여러 센서의 방향 비교

Practice/21_01_example.ipynb — 실습 4

```
sensors_4 = ["s_2", "s_3", "s_4", "s_7", "s_11"]
```

```
for sensor in sensors_4:
    첫값_4 = engine_4[sensor].iloc[0]
    끝값_4 = engine_4[sensor].iloc[-1]
    r_4 = engine_4["time_cycles"].corr(engine_4[sensor])
    방향_4 = "상승" if r_4 > 0 else "하강"
    print(f"{sensor:5} {첫값_4:>9.2f} -> {끝값_4:>9.2f} r={r_4:+.3f} {방향_4}")
```

```
▶ s_2      640.95 -> 646.57 r=+0.968 상승
▶ s_3     1585.27 -> 1605.90 r=+0.988 상승
▶ s_4     1399.71 -> 1416.77 r=+0.989 상승
▶ s_7      554.13 -> 550.10 r=-0.967 하강
▶ s_11      47.22 -> 51.25 r=+0.837 상승
```

STEP 1에서 예고한 정규화가 여기서 쓰입니다

```
# 센서마다 자릿수가 달라 한 축에 올리면 작은 센서가 눌립니다
```

```
# 0~1로 정규화해 "모양"만 남기면 방향 비교가 가능해집니다
```

```
plt.figure(figsize=(12, 5))
```

```
plt.title("정규화 후 이동평균(window=20) - 방향 비교 가능")
```

```
for sensor in sensors_4:
    ma_4 = engine_4[sensor].rolling(window=20).mean()
    정규화_4 = (ma_4 - ma_4.min()) / (ma_4.max() - ma_4.min())
    plt.plot(engine_4["time_cycles"], 정규화_4, label=sensor, linewidth=2)
```

```
plt.ylabel("0 ~ 1로 변환한 값")
```

```
plt.legend()
```

```
plt.show()
```

▶ 다섯 선이 0~1 범위에 함께 그려집니다

▶ s_7만 우하향이고 나머지 넷은 우상향입니다

설비 적용

다섯 센서 중 s_7만 하강하고 나머지 넷은 상승합니다. 방향이 같지만 **절댓값이 전부 0.837 이상**이라 다섯 센서 모두 열화 신호를 담고 있습니다.

여기서 중요한 것은 **"오른다"가 곧 "나쁘다"가 아니라는 점**입니다. 같은 상승도 센서에 따라 의미가 정반대일 수 있습니다. "온도가 오른다"는 사실이고, "온도가 올라 위험하다"는 해석입니다. **데이터는 사실까지, 해석은 그 센서의 의미를 아는 사람 몫**입니다.

정규화 $(x - \min) / (\max - \min)$ 은 **최솟값을 0, 최댓값을 1**로 만드는 변환입니다. 모양만 남기고 크기를 버리므로 **방향 비교에는 좋지만 크기 비교에는 쓸 수 없습니다**.

6-7. 사실과 해석을 구분하기

단계	내용	오늘 예	누가 하나
사실	그래프와 숫자에서 직접 읽히는 것	s_4가 회차와 r=+0.989로 상승	데이터
해석	그 사실이 무엇을 뜻하는지	해당 부위가 열화되고 있음	센서 의미를 아는 사람
판단	무엇을 할지	감시 주기를 줄이고 정비 계획	현장

개념

이동평균선의 방향은 사실이지만, 그 방향이 좋은지 나쁜지는 데이터가 말해 주지 않습니다. "온도가 오른다"는 사실이고 "온도가 올라 위험하다"는 해석입니다. 같은 상승도 센서에 따라 의미가 정반대일 수 있습니다.

그래서 **정상 방향을 아는 것이 중요합니다**. 어떤 센서는 원래 회차가 늘수록 오르는 게 정상이고, 어떤 센서는 내려가는 게 정상입니다. 정상 방향과 다르거나 **변화 속도가 너무 빠르면** 그때 주의 신호입니다.

오늘 데이터는 센서 이름이 s_2 s_3처럼 번호뿐이라 의미를 알 수 없습니다. 그래서 **방향 자체보다 "얼마나 일관되게 움직이는가"**를 봅니다. 실제 현장에서는 이 자리에 도메인 지식이 들어갑니다.

STEP 7

변동성 분석

rolling().std() · $\pm 2\sigma$ 밴드 · 점증과 급증

추세는 어디로 가는가만 알려줍니다. 값이 얼마나 불안한지는 말해 주지 않습니다. 그런데 **고장은 종종 평균보다 변동성으로 먼저 신호를 보냅니다.**

7-1. 추세만으로 부족한 이유

비유

잔잔한 호수와 파도치는 바다는 평균 수위가 같아도 완전히 다른 상태입니다. 매끈하게 오르는 설비와 심하게 출렁이며 오르는 설비도 마찬가지입니다. 추세만 보면 둘 다 "상승"이지만, **출렁이는 쪽이 훨씬 위험합니다.**

보는 것	질문	도구	오늘 데이터
추세	값이 어디로 가는가	<code>.rolling(20).mean()</code>	다섯 센서 모두 뚜렷
변동성	얼마나 불안한가	<code>.rolling(20).std()</code>	둘만 커짐

7-2. 표준편차 복습

교안 예시를 실제로 실행

```
# 표준편차 = 흩어진 정도 = 변동성을 재는 자
for vals in [[10, 10, 10, 10], [8, 12, 9, 11], [2, 18, 5, 15]]:
    print(vals, "-> std =", round(pd.Series(vals).std(), 2))

> [10, 10, 10, 10] -> std = 0.0
> [8, 12, 9, 11] -> std = 1.83
> [2, 18, 5, 15] -> std = 7.7
```

개념

표준편차는 단위가 원래 값과 같습니다. 그래서 "std 2도"는 "평균에서 2도쯤 흔들림"으로 직관적으로 읽힙니다. 그리고 값이 오르든 내리든 상관없이 **평균에서 얼마나 멀리 튀느냐만** 봅니다. 방향은 추세가, 폭은 변동성이 담당합니다.

7-3. 실습 5 — 이동표준편차

메서드 `.rolling().std()` 구간마다 표준편차를 계산

정의 이동평균과 한 쌍입니다. 창을 미끄러뜨리는 방식은 똑같고, 마지막에 평균 대신 표준편차를 낼 뿐입니다.

형태 `series.rolling(window=20).std()`

```
df_5 = pd.read_csv("Data/21_cmapss_fd001_sample.csv")
engine_5 = df_5[df_5["unit_nr"] == 2].sort_values("time_cycles")
```

```
std20_5 = engine_5["s_3"].rolling(window=20).std()
print(std20_5.describe().round(4))
```

```
▶ count      268.0000
▶ mean        1.0400
▶ std         0.2847
▶ min         0.5162
▶ 25%         0.8029
▶ 50%         1.0024
▶ 75%         1.2543
▶ max         1.8675
```

왜 쓰나 변동성이 시간에 따라 어떻게 변하는지 추적할 수 있습니다. 전체 표준편차 하나만 구하면 "이 센서는 얼마나 흔들리는가"만 알 수 있지만, 이동표준편차는 "언제부터 흔들리기 시작했는가"를 알려줍니다.

응용 전반부와 후반부를 갈라 평균을 비교하면 흔들림이 커졌는지 숫자로 판정할 수 있습니다.

```
n_5 = len(std20_5)
전반_5 = std20_5.iloc[: n_5 // 2].mean()
후반_5 = std20_5.iloc[n_5 // 2 :].mean()

print(f"전반부 평균 변동성: {전반_5:.4f}")
print(f"후반부 평균 변동성: {후반_5:.4f}"
      f"   ({(후반_5 / 전반_5 - 1) * 100:+.1f}%)"
      f"   ")
print("가장 흔들린 회차:", engine_5["time_cycles"].iloc[std20_5.argmax()],
      "| 값", round(std20_5.max(), 4))
```

```
▶ 전반부 평균 변동성: 0.8085
▶ 후반부 평균 변동성: 1.2394   (+53.3%)
▶ 가장 흔들린 회차: 278 | 값 1.8675
```

개념

count가 268인 것은 287에서 앞 19개(window - 1)를 뺀 값입니다. STEP 6에서 확인한 그 숫자가 여기서 다시 나옵니다. 그리고 .argmax()는 위치 번호를 주므로 .iloc과 짝지어야 합니다. 라벨을 주는 .idxmax()와 헷갈리면 엉뚱한 회차가 나옵니다.

설비 적용

후반부 변동성이 전반부보다 53.3% 커졌습니다. 그리고 가장 흔들린 지점이 278회차인데, 이 엔진은 287회차에서 고장났습니다. 고장 9회차 전에 변동성이 최대였던 것입니다. 값 자체가 오르는 것(추세)과 흔들림이 커지는 것(변동성)은 다른 신호이고, 이 경우 변동성이 고장을 더 가까이서 예고했습니다.

7-4. 실습 6 — $\pm 2\sigma$ 밴드

이동평균선 위아래로 표준편차 두 배만큼 띠를 둘러 그린 것입니다. 가운데 선은 추세, 띠의 폭은 변동성을 보여줍니다.

Practice/21_01_example.ipynb — 실습 6

```
ma_6 = engine_6["s_3"].rolling(window=20).mean()
std_6 = engine_6["s_3"].rolling(window=20).std()

# 평균 ± 2 × 표준편차
top_6 = ma_6 + 2 * std_6
bottom_6 = ma_6 - 2 * std_6

plt.figure(figsize=(12, 5))
plt.title("2번 엔진 s_3 - 이동평균과 ±2σ 정상 범위 밴드")
plt.plot(engine_6["time_cycles"], engine_6["s_3"], label="원본(raw)", alpha=0.5)
plt.plot(engine_6["time_cycles"], ma_6, label="이동평균 w=20", linewidth=2,
color="red")
plt.fill_between(engine_6["time_cycles"], bottom_6, top_6, alpha=0.25, label="±2σ
밴드")
plt.legend()
plt.show()

# 밴드를 벗어난 점을 이상 후보로 세어보기
밖_6 = (engine_6["s_3"] > top_6) | (engine_6["s_3"] < bottom_6)
유효_6 = ma_6.notna().sum()
print("밴드 계산이 가능한 구간:", 유효_6, "회차")
print("밴드를 벗어난 점:", 밖_6.sum(), f"개 ({밖_6.sum() / 유효_6 * 100:.1f}%)")
```

▶ 밴드 계산이 가능한 구간: 268 회차

▶ 밴드를 벗어난 점: 17 개 (6.3%)

함수 plt.fill_between() 두 선 사이를 색으로 채우기

정의 x와 아래·위 두 개의 y를 받아 그 사이 영역을 칠합니다. 밴드나 신뢰구간을 그릴 때 씁니다.

형태 plt.fill_between(x, 아래, 위, alpha=0.25)

```
plt.fill_between(engine_6["time_cycles"], bottom_6, top_6,  
                  alpha=0.25, label="±2σ 밴드")
```

▶ 이동평균선을 감싸는 반투명 띠가 그려집니다

왜 쓰나 선 두 개만 그리면 어디가 안이고 밖인지 한눈에 안 들어옵니다. 사이를 칠하면 "정상 범위"라는 개념이 그림으로 바로 전달됩니다.

응용 alpha를 반드시 낮춰야 합니다. 불투명하게 칠하면 밴드가 원본 선을 가려 버립니다.

```
# 밴드를 벗어난 점만 빨간 점으로 강조  
plt.scatter(engine_6.loc[밖_6, "time_cycles"],  
            engine_6.loc[밖_6, "s_3"],  
            color="red", s=30, zorder=5, label="밴드 밖")
```

▶ 벗어난 17개 점이 빨간색으로 표시됩니다

개념

zorder는 **그리는 순서(층)**입니다. 숫자가 클수록 위에 그려집니다. 밴드 위에 점을 얹으려면 밴드보다 큰 값을 줘야 합니다. 안 주면 점이 밴드 아래에 깔려 안 보일 수 있습니다.

주의

±2σ 밴드는 **관리도(control chart)의 기본 아이디어**입니다. 제조 현장에서 공정 관리에 쓰는 그 도구와 같은 개념이라, 앞으로 품질 관리 쪽 이야기가 나오면 여기서 배운 것이 그대로 이어집니다.

설비 적응

정규분포라면 ±2σ 밖은 약 5%가 정상입니다. 오늘 결과는 6.3%로 그보다 조금 많습니다. 즉 특별히 이상하다고 볼 정도는 아닙니다.

중요한 것은 **띠의 폭이 일정하지 않고 구간마다 넓어졌다 좁아진다**는 점입니다. 폭이 곧 변동성이므로, 띠가 넓어지는 구간이 불안정해지는 구간입니다. 실습 5에서 확인한 "후반부 53.3% 증가"가 그림에서는 **후반부로 갈수록 띠가 두꺼워지는 모습**으로 나타납니다.

7-5. 실습 7 — 점증과 급증

변동성이 커지는 방식도 두 가지입니다. **천천히 커지는지, 한순간에 튀는지에** 따라 대응이 달라집니다.

패턴	모양	원인	대응
점증	변동성이 서서히 우상향	점진적 마모·노화	정비 계획을 미리 세움
급증	어느 지점에서 가파른 솟구침	충격·이물·설정 변경	원본과 운영 기록을 함께 확인

Practice/21_01_example.ipynb — 실습 7

```
sensors_7 = ["s_2", "s_3", "s_4", "s_11"]

for sensor in sensors_7:
    std20_7 = engine_7[sensor].rolling(window=20).std()
    n_7 = len(std20_7)
    전반_7 = std20_7.iloc[: n_7 // 2].mean()
    후반_7 = std20_7.iloc[n_7 // 2 :].mean()
    증가율_7 = (후반_7 / 전반_7 - 1) * 100
    첨도_7 = std20_7.max() / std20_7.mean()

    if 증가율_7 > 20:
        패턴_7 = "점증 (후반 불안정)"
    elif 첨도_7 > 2.0:
        패턴_7 = "급증 (특정 시점 튀)"
    else:
        패턴_7 = "안정"

    print(f"{sensor:<6}{전반_7:>10.4f}{후반_7:>10.4f}"
          f"{증가율_7:>9.1f}%{첨도_7:>10.2f} {패턴_7}")
```

```
▶ 센서   전반부   후반부   증가율   최대/평균   패턴
▶ -----
▶ s_2      0.4644    0.4523    -2.6%     1.46   안정
▶ s_3      0.8085    1.2394    53.3%     1.80   점증 (후반 불안정)
▶ s_4      0.8535    0.8063    -5.5%     1.40   안정
▶ s_11     0.1210    0.1995    64.9%     2.78   점증 (후반 불안정)
```

센서	추세 상관	변동성 증가율	최대/평균	종합 판정
s_2	+0.968	-2.6%	1.46	추세만 — 안정적 열화

센서	추세 상관	변동성 증가율	최대/평균	종합 판정
s_3	+0.988	+53.3%	1.80	추세 + 변동성
s_4	+0.989	-5.5%	1.40	추세만 — 안정적 열화
s_11	+0.837	+64.9%	2.78	추세 + 변동성 + 급증

설비 적용

오늘 문서에서 가장 중요한 표입니다. 추세만 보면 네 센서가 다 비슷합니다. 상관계수가 전부 0.837 이상이라 모두 "뚜렷한 상승"입니다.

그런데 변동성을 보면 완전히 같습니다. s_2와 s_4는 오히려 후반에 더 안정적이 되었고(-2.6%, -5.5%), s_3과 s_11만 크게 흔들리기 시작했습니다(+53.3%, +64.9%).

특히 s_11은 최대/평균이 2.78로 특정 시점에 크게 튼 흔적까지 있습니다. 추세와 변동성이 둘 다 나쁜 센서이므로, 감시 우선순위를 매긴다면 s_11 → s_3 → 나머지 순입니다.

"추세만 보고 다 위험하다"고 하면 우선순위가 없어집니다. 변동성을 함께 봐야 어디를 먼저 볼지 정해집니다. 이것이 오늘 두 도구를 나란히 배운 이유입니다.

개념 최대 / 평균 (첨도 대응) 급증 여부를 재는 간단한 지표

정의 이동표준편차의 최댓값을 평균으로 나눈 값입니다. 특정 시점에만 크게 튀었으면 이 비율이 커집니다.

형태 `std20.max() / std20.mean()`

```
# 네 센서의 최대/평균 비교
for sensor in ["s_2", "s_3", "s_4", "s_11"]:
    st = engine_7[sensor].rolling(20).std()
    print(f"{sensor:5} 최대 {st.max():.4f} / 평균 {st.mean():.4f}"
          f" = {st.max()/st.mean():.2f}")
```

- ▶ s_2 최대 0.6697 / 평균 0.4579 = 1.46
- ▶ s_3 최대 1.8675 / 평균 1.0400 = 1.80
- ▶ s_4 최대 1.1603 / 평균 0.8281 = 1.40
- ▶ s_11 최대 0.4542 / 평균 0.1631 = 2.78

왜 쓰나 증가율만으로는 급증을 못 잡습니다. 전반·후반 평균이 비슷해도 중간에 한 번 크게 튀었을 수 있기 때문입니다. 두 지표를 함께 봐야 점증과 급증을 구분할 수 있습니다.

응용 실습 코드는 증가율 > 20이면 점증, 아니면서 최대/평균 > 2.0이면 급증으로 판정합니다. 판정 기준은 데이터마다 조정해야 하는 값입니다.

```
# 언제 튀었는지까지 확인
st = engine_7["s_11"].rolling(20).std()
print("최대 시점:", engine_7["time_cycles"].iloc[st.argmax()], "회차")
print("전체 회차:", engine_7["time_cycles"].max())
```

▶ 최대 시점: 275 회차

▶ 전체 회차: 287

개념

s_11의 변동성 최댓값이 **275회차**, s_3은 **278회차**입니다. 둘 다 **고장(287회차) 직전 10여 회차** 구간에 몰려 있습니다. 어제 배운 "변동성은 중간 단계의 신호"가 여기서 확인됩니다.

자주 하는 실수

급증이 항상 고장을 뜻하지는 않습니다. 운전 조건을 바꿨거나 정비를 했거나 센서를 교체했을 수도 있습니다. **원본 그래프와 운영 기록을 함께 보고** 원인을 판단해야 합니다.

7-6. 오늘 데이터가 말해 준 것



개념

① **추세 판정** — 다섯 센서 모두 $|r|$ 0.837 이상으로 뚜렷합니다. s_7만 하강이고 나머지는 상승입니다. 여기까지는 "다 열화 중"이라는 것 외에 정보가 없습니다.

② **변동성 판정** — s_3(+53.3%)과 s_11(+64.9%)만 후반에 크게 흔들립니다. s_2와 s_4는 오히려 안정적입니다. **여기서 우선순위가 생깁니다.**

③ **시점 확인** — 두 센서의 변동성 최댓값이 275회차와 278회차입니다. 고장은 287회차입니다. **고장 9~12회차 전에 변동성이 정점을 찍었습니다.**

결론은 "추세는 오래전부터 있었지만, 변동성은 **마지막에 커졌다**"입니다. 어제 배운 신호 강도 순서 — 추세가 약한 조기 신호, 변동성이 중간 신호 — 가 실제 데이터에서 확인된 셈입니다.

치트시트

오늘 배운 것을 한 장으로

A-1. 시간 데이터 준비 3단계

복사해서 첫 셀에 붙여 두십시오

```
import pandas as pd
```

```
df = pd.read_csv("파일.csv")
```

① 글자 → 시간

```
df["timestamp"] = pd.to_datetime(df["timestamp"])
```

② 인덱스로 올리기 (결과를 반드시 다시 담기)

```
df = df.set_index("timestamp")
```

③ 시각 오름차순 정렬

```
df = df.sort_index()
```

확인

```
print(type(df.index).__name__)      # DatetimeIndex
```

```
print(df.index.is_monotonic_increasing)  # True
```

A-2. 시간 도구

도구	하는 일	예	주의
<code>pd.to_datetime()</code>	글자를 시간 값으로	<code>pd.to_datetime(df["ts"])</code>	맨 처음 한 번
<code>.dt.year</code>	시간 조각 꺼내기	<code>df["ts"].dt.hour</code>	괄호 없음
<code>.set_index()</code>	열을 인덱스로	<code>df.set_index("ts")</code>	<code>df</code> =로 다시 담기
<code>.sort_index()</code>	인덱스 정렬	<code>df.sort_index()</code>	기간 자르기 전제
<code>.loc[]</code>	시각으로 행 선택	<code>df.loc["2024-03-02"]</code>	끝을 포함
<code>pd.date_range()</code>	시각 목록 만들기	<code>pd.date_range(a, b, freq="1h")</code>	누락 점검용
<code>.resample()</code>	시간 단위 다시 묶기	<code>.resample("1D").mean()</code>	집계 함수 필수

도구	하는 일	예	주의
<code>.asfreq()</code>	간격만 재배치	<code>.asfreq("1h")</code>	누락 확인용
<code>.interpolate()</code>	앞뒤 이어 채우기	<code>s.interpolate()</code>	양 끝은 못 채움
<code>.diff()</code>	직전 값과의 차이	<code>s.diff()</code>	첫 값은 NaN

A-3. 빈도 문자열

글자	단위	대소문자	자주 쓰는 형태
min	분	소문자	"15min" "30min"
h	시간	소문자 (예전 H)	"1h" "6h" "12h"
D	일	대문자	"1D" "3D"
W	주	대문자	"W"
ME	월말	대문자	"ME"

A-4. 추세와 변동성 도구

도구	재는 것	코드	판정
이동평균	추세 — 어디로 가는가	<code>.rolling(20).mean()</code>	선의 방향
이동표준편차	변동성 — 얼마나 불안한가	<code>.rolling(20).std()</code>	전반 대비 후반 증가율
상관계수	추세의 직선성	<code>df["time_cycles"].corr(df[s])</code>	부호 = 방향, $ r $ = 강도
$\pm 2\sigma$ 밴드	정상 범위	<code>ma $\pm$ 2 * std</code>	밖의 점 = 이상 후보
최대/평균	급증 여부	<code>std.max() / std.mean()</code>	2.0 넘으면 급증

A-5. 윈도우 크기 고르기

윈도우	선 모양	반응 속도	앞쪽 빈 구간	언제
작게 (5~10)	거칠게 떨림	빠름	적음	짧은 변화를 잡을 때
적당히 (10~30)	적절	적절	적절	교안 권장 — 192주기 기준
크게 (50 이상)	매끈함	느림	287 중 49 손실	긴 흐름만 볼 때

개념

NaN 개수는 항상 window - 1개입니다. 그리고 유효 구간은 전체 - (window - 1)입니다. 오늘 데이터에서 287회차에 윈도우 20을 쓰면 268회차가 유효 구간이 되는데, 이 숫자가 이동표준편차의 count와 $\pm 2\sigma$ 밴드의 유효 구간에 반복해서 나왔습니다. 한 번 계산해 두면 여러 곳에서 계산에 쓸 수 있습니다.

A-6. 오늘 나온 숫자 요약

항목	값	어디서
timestamp 샘플 크기	182행 6열	STEP 2
기간	2024-03-01 00시 ~ 03-08 23시	STEP 3
누락 시각	10개 (이론 192 - 실제 182)	STEP 3
일별 평균 추세	641.07 → 642.35 (8일 단조 증가)	STEP 5
업샘플링 결측	30분 단위 383행 중 201개 → 보간 후 0	STEP 5
cycle 샘플 크기	1031행 10열 · 엔진 5대	STEP 6
엔진별 수명	155 ~ 287 회차 (1.85배 차이)	STEP 1
2번 엔진 추세	s_4 1399.71 → 1416.77 · r=+0.989	STEP 6
이동평균 효과	std 5.084 → 4.875 (w=10)	STEP 6
변동성 증가	s_3 +53.3% · s_11 +64.9%	STEP 7
변동성 최댓값 시점	275 · 278회차 (고장 287회차)	STEP 7
$\pm 2\sigma$ 밴드 밖	268 중 17개 (6.3%)	STEP 7

주의

표에 있는 숫자는 전부 이 두 파일에서만 나온 값입니다. 어제 쓴 20_cmapss 샘플과 오늘의 21_cmapss 샘플은 4·5번 엔진이 다르므로 숫자를 섞어 쓰면 안 됩니다. 계산할 때는 파일 이름 앞의 번호를 먼저 확인하십시오.

오류·함정 사전

X 이렇게 하면 → ✓ 바로잡기

B-1. 자주 만나는 오류 열두 가지

X 이렇게 하면	무슨 일이	✓ 바로잡기
<code>df["ts"].dt.year</code>	변환 전이면 <code>AttributeError</code>	<code>pd.to_datetime()</code> 먼저
<code>df.set_index("ts")</code>	<code>df</code> =가 없으면 원본이 그대로	<code>df = df.set_index("ts")</code>
정렬 없이 기간 자르기	오류 또는 빈 결과	<code>.sort_index()</code>
<code>.resample("H")</code>	<code>FutureWarning</code> — 대문자 폐기	<code>.resample("h")</code>
<code>.resample("T")</code>	<code>FutureWarning</code> — 대문자 폐기	<code>.resample("min")</code>
<code>.resample("1D")</code>	집계 함수 없으면 객체만 남음	<code>.resample("1D").mean()</code>
<code>.rolling(20)</code>	집계 함수 없으면 객체만 남음	<code>.rolling(20).mean()</code>
<code>.loc[0]</code>	필터 후 인덱스가 0이 아님	<code>.iloc[0]</code>
<code>.iloc[first_valid_index()]</code>	라벨을 위치로 써서 엉뚱한 행	<code>.loc[first_valid_index()]</code>
<code>.iloc[idxmax()]</code>	라벨을 위치로 씀	<code>.iloc[argmax()]</code>
<code>(s.diff() > 0).all()</code>	첫 NaN 때문에 항상 <code>False</code>	<code>(s.diff().dropna() > 0).all()</code>
맨 앞 결측에 <code>interpolate</code>	채워지지 않고 그대로 NaN	<code>limit_direction="both"</code>

B-2. 헷갈리는 짝들

A	B	차이	오늘 예
<code>.loc[]</code>	<code>.iloc[]</code>	라벨 vs 위치	필터 후 첫 행은 <code>.iloc[0]</code>
<code>.idxmax()</code>	<code>.argmax()</code>	라벨 vs 위치	<code>time_cycles.iloc[argmax()]</code>
<code>.resample()</code>	<code>.asfreq()</code>	묶어 집계 vs 재배치	641.058 vs 641.17
<code>.rolling()</code>	<code>.resample()</code>	개수 단위 vs 시간 단위	창 20개 vs 하루
<code>.ffill()</code>	<code>.interpolate()</code>	복사 vs 직선 보간	<code>[10,10,10,40]</code> vs <code>[10,20,30,40]</code>

A	B	차이	오늘 예
.dt	.rolling()	속성 vs 메서드	괄호 없음 vs 있음

개념

.rolling()과 .resample()은 이름이 비슷하지만 기준이 다릅니다. rolling(20)은 **값 20개**를 묶고, resample("1D")은 **하루 안의 값 전부**를 묶습니다. 하루에 24개가 들어 있을 수도 20개가 들어 있을 수도 있습니다. cycle 데이터처럼 시간 인덱스가 없으면 rolling만 쓸 수 있고, timestamp 데이터에서는 둘 다 쓸 수 있습니다.

B-3. 그림은 맞는데 결론이 틀리는 경우

함정	왜 위험한가	대응
행 개수를 당연하게 믿기	하루가 24행일 거라 가정 — 실제로는 20행	date_range
첫값·끝값만으로 추세 판정	s_14는 첫끝은 변했는데 $r=+0.298$	상관계수로 계산
스케일 다른 센서 겹쳐 그리기	8138 옆의 38은 안 보임 — 실제로는 s_20이 더 변함	정규화 또는 따로 그리기
추세만 보고 위험도 판단	네 센서 모두 $ r 0.837$ 이상 — 구분 안 됨	변동성을 함께
큰 윈도우가 좋다고 생각	w=50이면 앞 49회차 를 통째로 손실	목적에 맞는 크기
보간 후 변동성 재기	직선으로 채워 흔들림이 사라짐	보간 전에 변동성 계산
세로축 범위 안 보고 판단	8일 변화가 겨우 1.29°C	축 눈금을 반드시 확인

주의

B-1은 오류 메시지가 뜨는 실수라 금방 알아챱니다. B-3은 코드가 멀쩡히 돌아가고 그림도 잘 나오는데 결론만 틀리는 실수라 훨씬 위험합니다. 오늘 데이터에서 가장 극적인 예는 **"하루가 24행이 아니다"**입니다. 10개 시각이 빠져 있는데 이걸 모르고 하루 24행을 전제로 계산하면 모든 결과가 조용히 어긋납니다.

실습 하이라이트와 다음 시간

오늘의 결론 · 예고

C-1. 실습 한눈에

교안	실습	한 일	핵심 결과
20_01	5	첫걸 값으로 추세 판정	s_11 +0.88 상승 · s_7 -7.67 하강
	6	센서 시계열 그리기	우상향·우하향 곡선 확인
	7	스케일 문제 체험	s_14 8138 vs s_20 38
	8	구조 미니 리포트	1116행 26열 · 엔진 5대 · 상수 8개
20_02	1	문자열 날짜 변환	object → datetime64[ns]
	2	시간 조각 추출	.dt.hour → 파생 열 → groupby
	3	시간 인덱스 만들기	DatetimeIndex · 3/1 0시~3/8 23시 · 182행
	4	기간 자르기	3/2 20행 · 3/5~7 70행 · 누락 10개 발견
	5	리샘플링 집계	6h 첫 구간 평균 641.058 · 최대 641.18
	6	단위별 행 수	1h 192 · 6h 32 · 12h 16 · 1D 8
	7	일별 추세 확인	641.07 → 642.35 · 8일 단조 증가
	8	업샘플링과 보간	30min 결측 201 → 0
21_01	1	센서 시계열	s_4 1399.71 → 1416.77 · r=+ 0.989
	2	이동평균	std 5.084 → 4.875 · NaN 9개
	3	윈도우 비교	w=5/20/50 · NaN 4/19/49
	4	여러 센서 방향	s_7만 하강 · 나머지 상승
	5	이동표준편차	후반 + 53.3% · 최대 278회차
	6	$\pm 2\sigma$ 밴드	유효 268 · 밖 17개(6.3%)
	7	점증과 급증	s_3 +53.3% · s_11 + 64.9%

C-2. 오늘 데이터가 말해 준 것

열아홉 개의 실습을 흩어진 조각이 아니라 **하나의 흐름**으로 다시 읽어 보겠습니다.

설비 적용

- ① **시간 데이터를 다룰 준비를 갖췄습니다.** 문자열 날짜를 시간으로 바꾸고, 인덱스로 올리고, 정렬했습니다. 이 세 줄이 있어야 나머지 모든 도구가 작동합니다.
- ② **데이터의 숨은 결함을 찾았습니다.** 하루가 24행이 아니라 20행이었고, `date_range`와 비교해 보니 **10개 시각이 통째로 빠져** 있었습니다. `resample("1h")`이 182행을 192행으로 늘린 것이 그 증거였습니다.
- ③ **잡음을 걷어내고 추세를 봤습니다.** 시간별 원본은 위아래로 튀지만 일별 평균은 **641.07에서 642.35로 8일 내내 단조 증가**했습니다. 변화량은 겨우 1.29°C라 축 범위를 좁혀야 보이는 크기였습니다.
- ④ **추세와 변동성이 다른 신호임을 확인했습니다.** 네 센서 모두 추세는 $|r|$ 0.837 이상으로 뚜렷한데, 변동성은 `s_3`(+53.3%)과 `s_11`(+64.9%)만 커졌습니다. **추세만 보면 우선순위가 안 생기고, 변동성을 봐야 어디를 먼저 볼지 정해집니다.**
- ⑤ **신호의 시점을 확인했습니다.** 두 센서의 변동성 최댓값이 **275회차와 278회차**, 고장은 **287회차**였습니다. 고장 9~12회차 전에 변동성이 정점을 찍었습니다.

C-3. 24~27일차 네 회차의 연결



회차	배운 것	오늘 어디서 다시 쓰었나
21일차	<code>corr()</code>	추세 방향과 강도 판정
22·23일차	<code>isna()</code> · <code>fillna</code> 계열	<code>interpolate()</code> 로 시계열 결측 처리
24일차	<code>quantile()</code> · 이상치 경계	$\pm 2\sigma$ 밴드 — 같은 발상의 다른 기준
25일차	<code>plt.plot</code> · <code>fill_between</code>	원본과 이동평균 겹쳐 그리기
26일차	시계열 개념 · <code>sort_values</code>	오늘 전체의 전제
27일차	<code>resample</code> · <code>rolling</code>	추세와 변동성을 직접 측정

연결

24일차의 IQR 이상치와 오늘의 $\pm 2\sigma$ 밴드는 발상이 같습니다. 둘 다 "정상 범위를 정하고 그 밖을 이상 후보로 본다"는 구조입니다. 차이는 IQR은 전체에 대해 한 번, $\pm 2\sigma$ 밴드는 구간마다 계속 계산한다는 점입니다. 시계열에서는 정상 범위 자체가 시간에 따라 변하므로 후자가 더 적합합니다.

C-4. 다음 시간 예고

오늘 마지막에 주기성 이야기를 짧게 하셨습니다. 다음 시간에 21_02부터 이어집니다.

순서	교안	핵심 도구	무엇을 하나
① 21_02	변화량과 주기 분석	<code>diff pct_change</code>	급변 지점 찾기 · 반복 패턴
② 21_03	다중 센서 상태 변화	<code>corr heatmap</code>	여러 센서를 함께 보기
③ 22장	측정 주기와 리샘플링 후 결측	<code>asfreq interpolate</code>	오늘 도구의 심화

개념

`.diff()`는 오늘 이미 맛봤습니다. 실습 7에서 일별 평균이 매일 올랐는지 확인할 때 썼던 그 메서드가 다음 시간의 주인공이 됩니다. 그리고 주기성은 "얼마만의 길이마다 같은 패턴이 반복되는가"를 보는 것으로, 하루 주기·일주일 주기처럼 설비 운전에도 반복 패턴이 있습니다. 오늘 배운 `resample`이 주기를 재는 기본 도구가 됩니다.

미리보기

```
# 다음 시간 미리보기 — 오늘 도구 위에 얹힙니다
engine = df[df["unit_nr"] == 2].sort_values("time_cycles")

# 변화량 — 직전 회차 대비
engine["변화량"] = engine["s_4"].diff()

# 변화율 — 몇 퍼센트 변했나
engine["변화율"] = engine["s_4"].pct_change() * 100

# 급변 지점 — 변화량이 평소보다 큰 회차
임계 = engine["변화량"].std() * 2
print("급변 후보:", (engine["변화량"].abs() > 임계).sum(), "회차")
```

▶ 급변 후보: 14 회차

오늘의 성취와 복습 루틴

오늘 할 수 있게 된 것

아래 여섯 가지를 코드를 보지 않고 할 수 있다면 오늘 수업은 소화한 것입니다.

#	할 수 있게 된 것	확인 방법
1	시간 데이터 준비 3단계를 외워서 쓴다	변환 → 인덱스 → 정렬 순서를 아나
2	시각으로 기간을 잘라낸다	<code>.loc["a":"b"]</code> 가 끝을 포함하는 걸 아나
3	누락된 시각을 찾아낸다	<code>date_range</code> 와 <code>difference</code> 를 쓸 수 있나
4	시간 단위를 바꿔 추세를 본다	<code>resample("1D").mean()</code> 을 바로 쓸 수 있나
5	이동평균으로 잡음을 걷어낸다	NaN이 window-1개인 이유를 아나
6	추세와 변동성을 구분해 읽는다	둘이 왜 다른 신호인지 설명할 수 있나

복습 루틴 — 3일 계획

시점	할 일	시간
오늘 저녁	20_engine01_timestamp_sample.csv로 준비 3단계를 코드 안 보고 다시	25분
내일	2번이 아닌 다른 엔진을 골라 이동평균·이동표준편차를 다시 계산	30분
주말	엔진 다섯 대의 변동성 증가율을 비교해 어느 엔진이 가장 불안정한지 판정	40분

주의

주말 과제가 특히 좋은 연습입니다. 엔진마다 수명이 155~287회차로 달라 회차 수가 다른 데이터를 어떻게 공평하게 비교할지 스스로 정해야 합니다. 실무에서 늘 만나는 종류의 문제입니다.

오늘 배운 도구는 앞으로 모든 시계열 분석의 기본기가 됩니다. timestamp 데이터를 만나면 변환 → 인덱스 → 정렬 → 집계 순서로 접근하면 되고, 어떤 데이터든 추세와 변동성을 나눠 보는 습관이 분석의 깊이를 가릅니다.

수고하셨습니다.